Laboratorio de Ingeniería del Conocimiento con Owl/Protégé

Universidad de Valladolid
Escuela de Ingeniería Informática
Laboratorio de Ingeniería de Conocimiento
Curso 2025-2026

SEGUNDA SESION

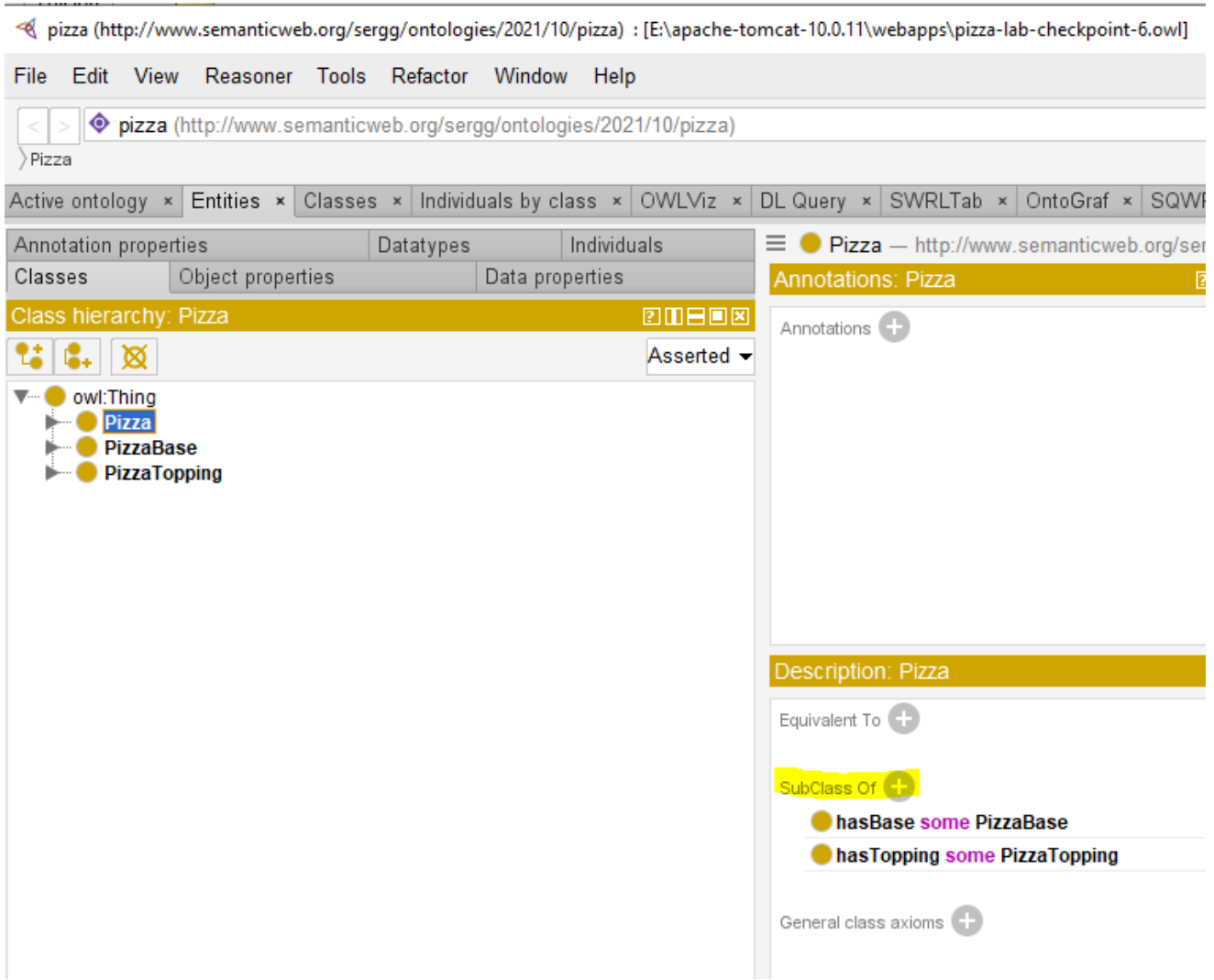
Sesión 2 - Contenido

- Definiciones de clases más completas.
- Funcionamiento del razonador.
- Clases Definidas y Primitivas.
- Clases Enumeradas.
- Restricciones sobre propiedades y datos.
- Definición de propiedades transitivas.

Definiciones de clases más completas

Descripción de la clase mediante restricciones

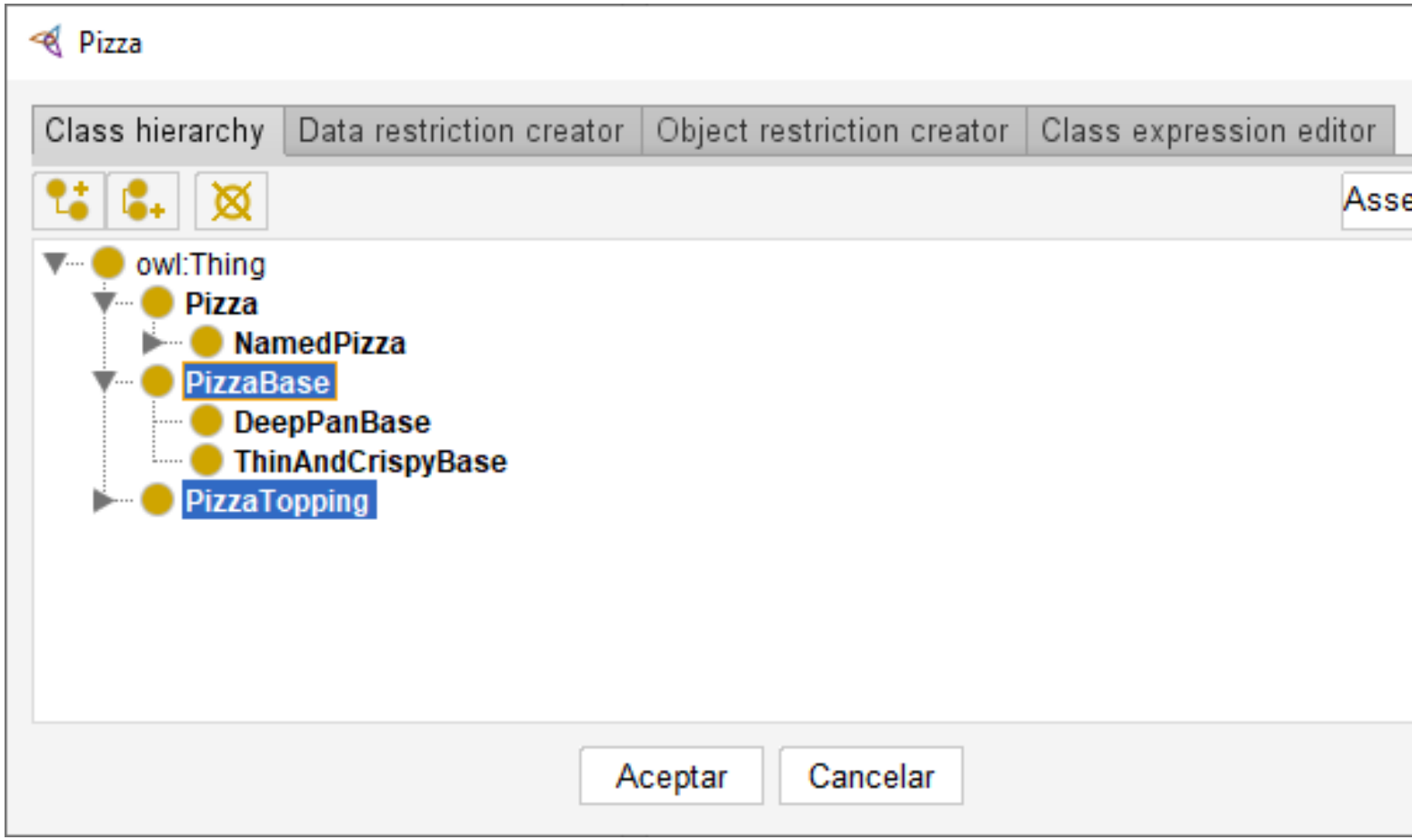
- Las restricciones permiten describir las propiedades de los elementos de la clase.
- En el menú de Classes, en la ventana Descripcion, bajo Subclass Of, vemos las restricciones que ya hemos añadido.
- Al hacer clic en el + de Subclass Of, se abre el siguiente menú, con 4 pestañas (ver página siguiente)



Mecanismos para especificar restricciones para una subclase (I)

- **Class hierarchy**: permite seleccionar una o más clases para introducir la restricción de que la subclase deberá heredar de esas clases.

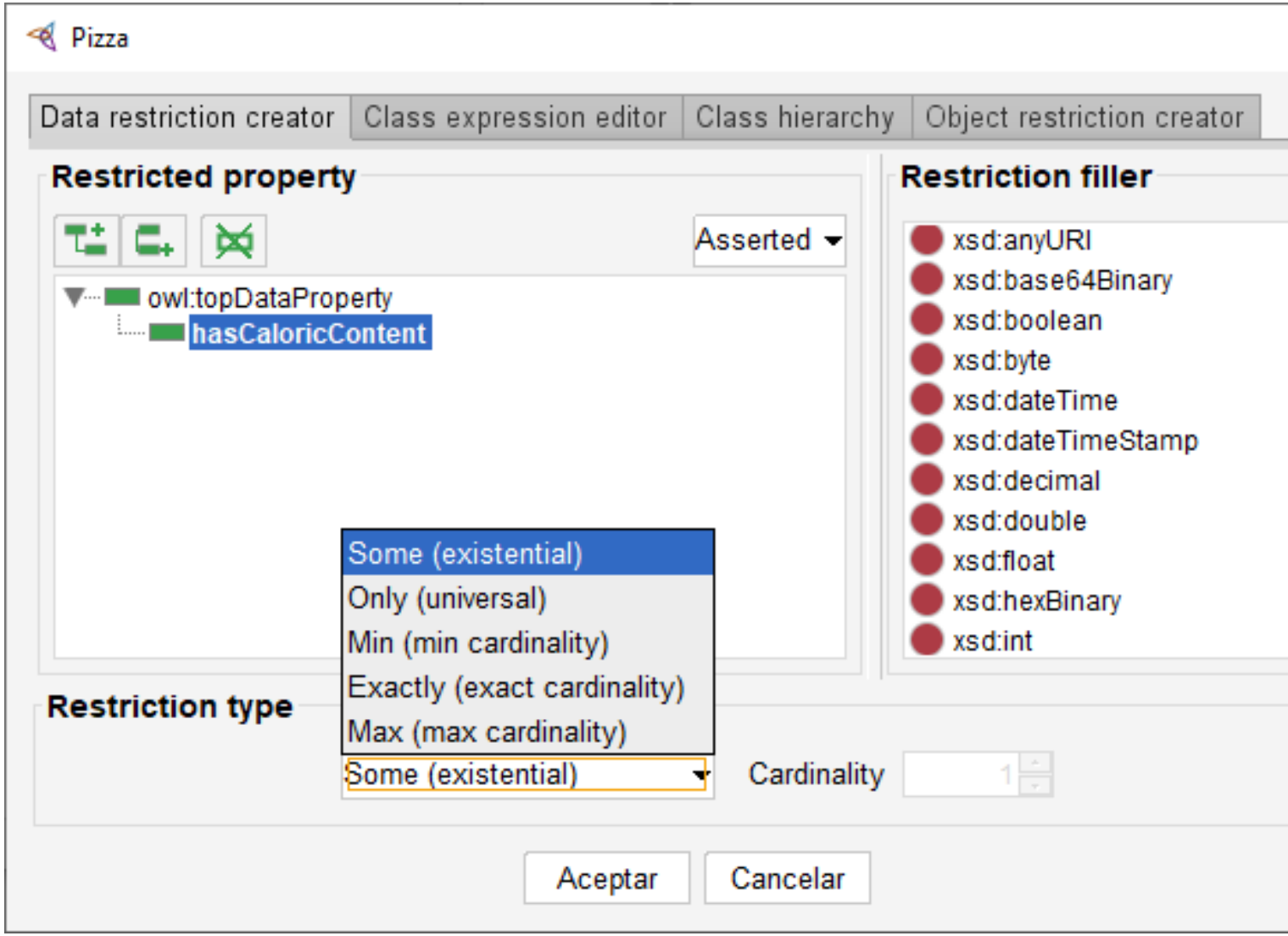
- No hacerlo en este ejemplo.



Mecanismos para especificar restricciones para una subclase (II)

- **Data restriction creator**: permite establecer una restricción sobre propiedades de datos.

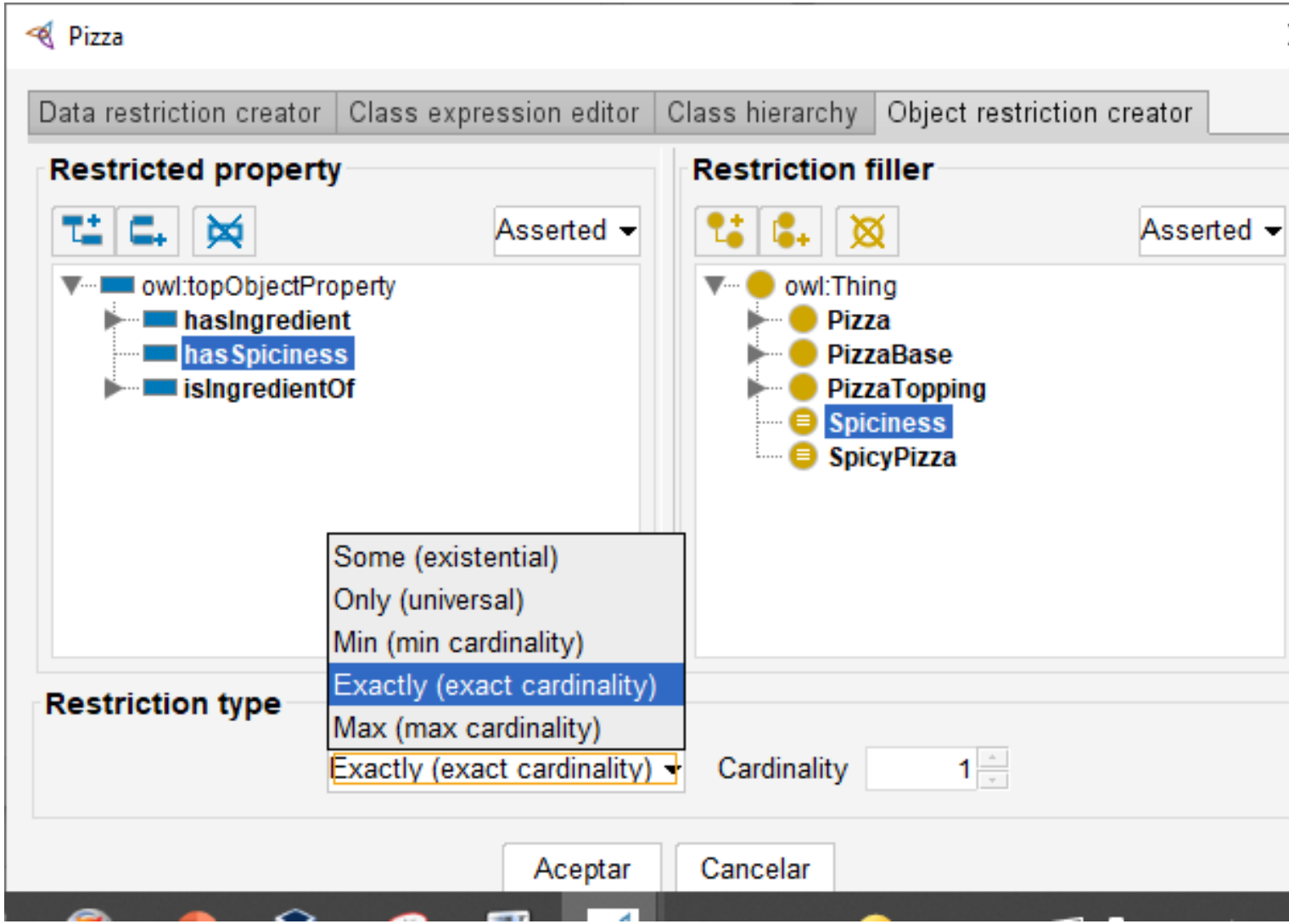
- Pueden ser de varios tipos:
  - Existencial: Some.
  - Universal: Only.
  - De Cardinalidad: Min, Exactly, Max.



Mecanismos para especificar restricciones para una subclase (III)

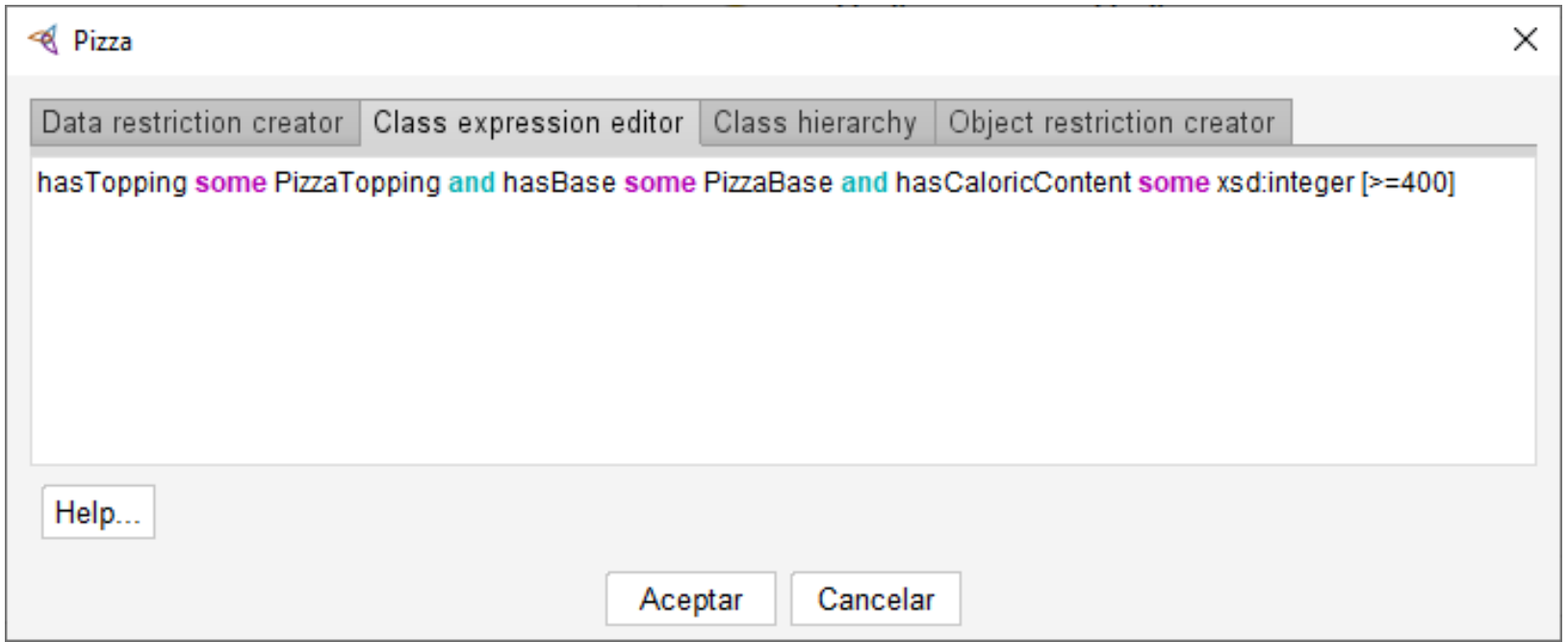
- **Object restriction creator**: permite establecer una restricción sobre propiedades de objetos.

- Pueden ser de varios tipos:
  - Existencial: Some.
  - Universal: Only.
  - De Cardinalidad: Min, Exactly, Max.



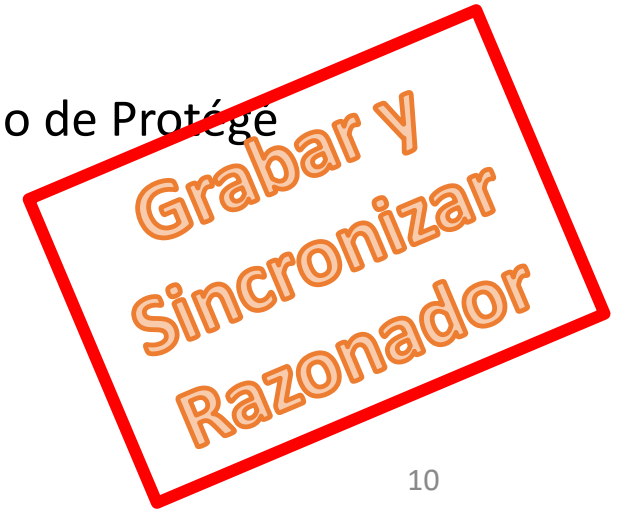
Mecanismos para especificar restricciones para una subclase (IV)

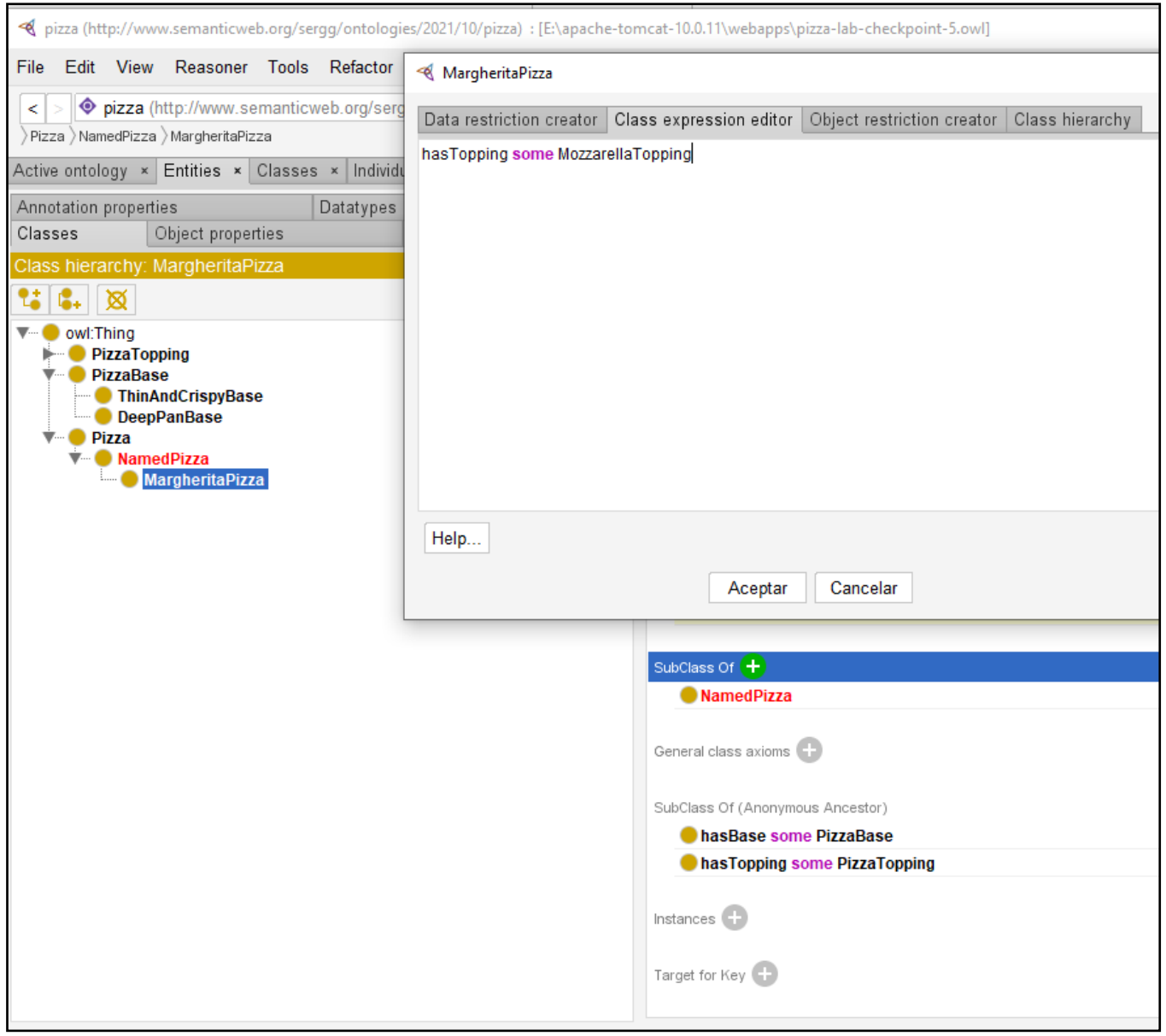
- **Class expression editor**: permite elaborar reglas más completas usando una expresión lógica.
- Para conocer más acerca de la sintaxis de las expresiones que podemos incluir, hacer clic en el botón ayuda o ir a http://protegeproject.github.io/protege/class-expression-syntax/



Creación de Subclases de pizza

- Creamos una jerarquía de clases bajo Pizza (ver siguiente página).
- Clase NamedPizza como subclase de Pizza (que representará a pizzas con un nombre conocido que describiremos en la anotación).
- A continuación, creamos la clase MargheritaPizza como subclase de NamedPizza (añadir anotación con un texto descriptivo).
- Seleccionando la clase MargheritaPizza, Seleccionar Subclass Of +
  - Class expression
    - hasTopping some Mo… (<CTRL>+<Space>) → para usar el autorrelleno de Protégé
      - …MozzarellaTopping
  - Repetir con TomatoTopping.
- Observar en el interfaz de Protégé las inferencias.

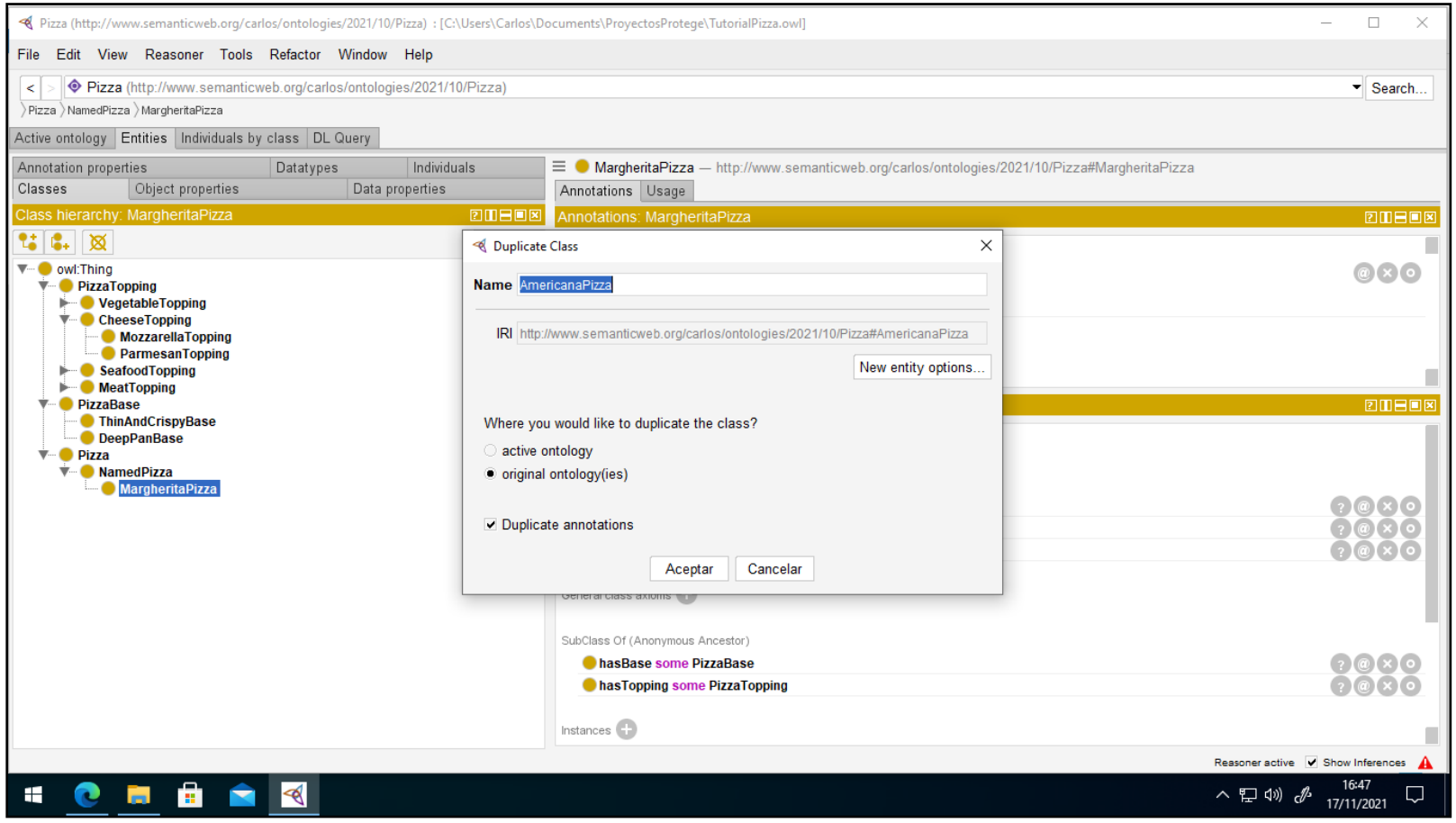


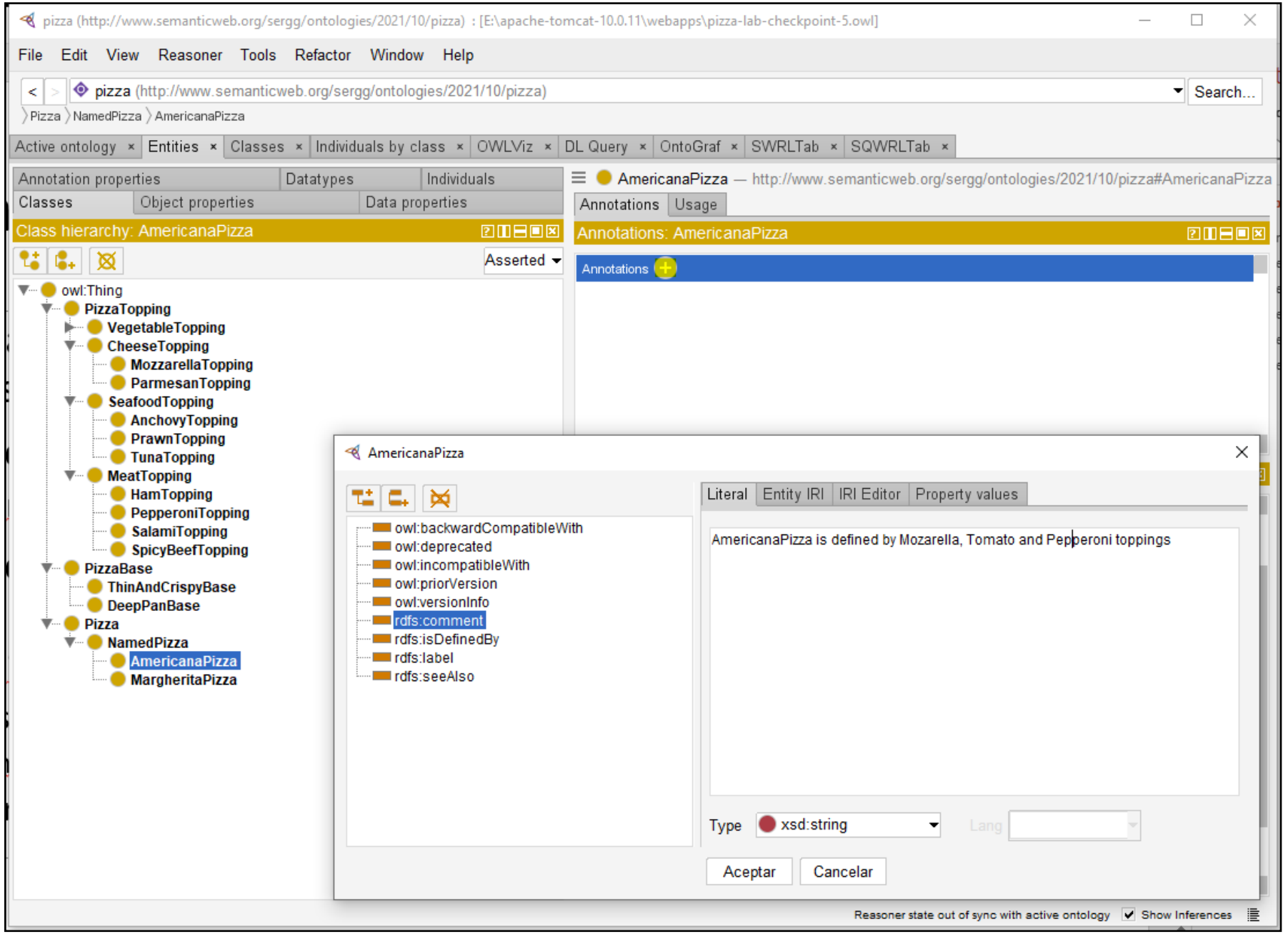

pizza (http://www.semanticweb.org/sergg/ontologies/2021/10/pizza) : [E:\apache-tomcat-10.0.11\webapps\pizza-lab-checkpoint-5.owl]
File Edit View Reasoner Tools Refactor
pizza (http://www.semanticweb.org/serg
Pizza NamedPizza MargheritaPizza
Active ontology Entities Classes Individu
Annotation properties Datatypes
Classes Object properties
Class hierarchy: MargheritaPizza
owl:Thing
PizzaTopping
PizzaBase
ThinAndCrispyBase
DeepPanBase
Pizza
NamedPizza
MargheritaPizza
MargheritaPizza
Data restriction creator Class expression editor Object restriction creator Class hierarchy
hasTopping some MozzarellaTopping
Help...
Aceptar Cancelar
SubClass Of
NamedPizza
General class axioms
SubClass Of (Anonymous Ancestor)
hasBase some PizzaBase
hasTopping some PizzaTopping
Instances
Target for Key

Clonar clases

- Creamos la clase AmericanaPizza clonando la clase Margherita y añadiendo las restricciones adicionales Seleccionando la clase MargheritaPizza. *Right Click > Duplicate class (ver página siguiente)*
  - Name: AmericanaPizza
- Seleccionando la clase AmericanaPizza y añadimos una restricción adicional
  - SubClass Of +
  - Class expression
    - hasTopping some PepperoniTopping
- Añadir una anotación a la clase con la descripción de esta pizza *(ver dos páginas más adelante)*



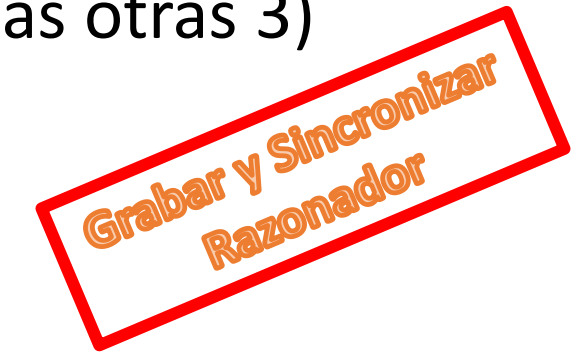

Pizza (http://www.semanticweb.org/carlos/ontologies/2021/10/Pizza) : [C:\Users\Carlos\Documents\ProyectosProtege\TutorialPizza.owl]
File Edit View Reasoner Tools Refactor Window Help
Pizza (http://www.semanticweb.org/carlos/ontologies/2021/10/Pizza)
Search...
Pizza NamedPizza MargheritaPizza
Active ontology Entities Individuals by class DL Query
Annotation properties Datatypes Individuals
Classes Object properties Data properties
Class hierarchy: MargheritaPizza
owl:Thing
PizzaTopping
VegetableTopping
CheeseTopping
MozzarellaTopping
ParmesanTopping
SeafoodTopping
MeatTopping
PizzaBase
ThinAndCrispyBase
DeepPanBase
Pizza
NamedPizza
MargheritaPizza
MargheritaPizza — http://www.semanticweb.org/carlos/ontologies/2021/10/Pizza#MargheritaPizza
Annotations Usage
Annotations: MargheritaPizza
Duplicate Class
Name AmericanaPizza
IRI http://www.semanticweb.org/carlos/ontologies/2021/10/Pizza#AmericanaPizza
New entity options...
Where you would like to duplicate the class?
active ontology
original ontology(ies)
Duplicate annotations
Aceptar Cancelar
SubClass Of (Anonymous Ancestor)
hasBase some PizzaBase
hasTopping some PizzaTopping
Instances
Reasoner active Show Inferences
16:47
17/11/2021

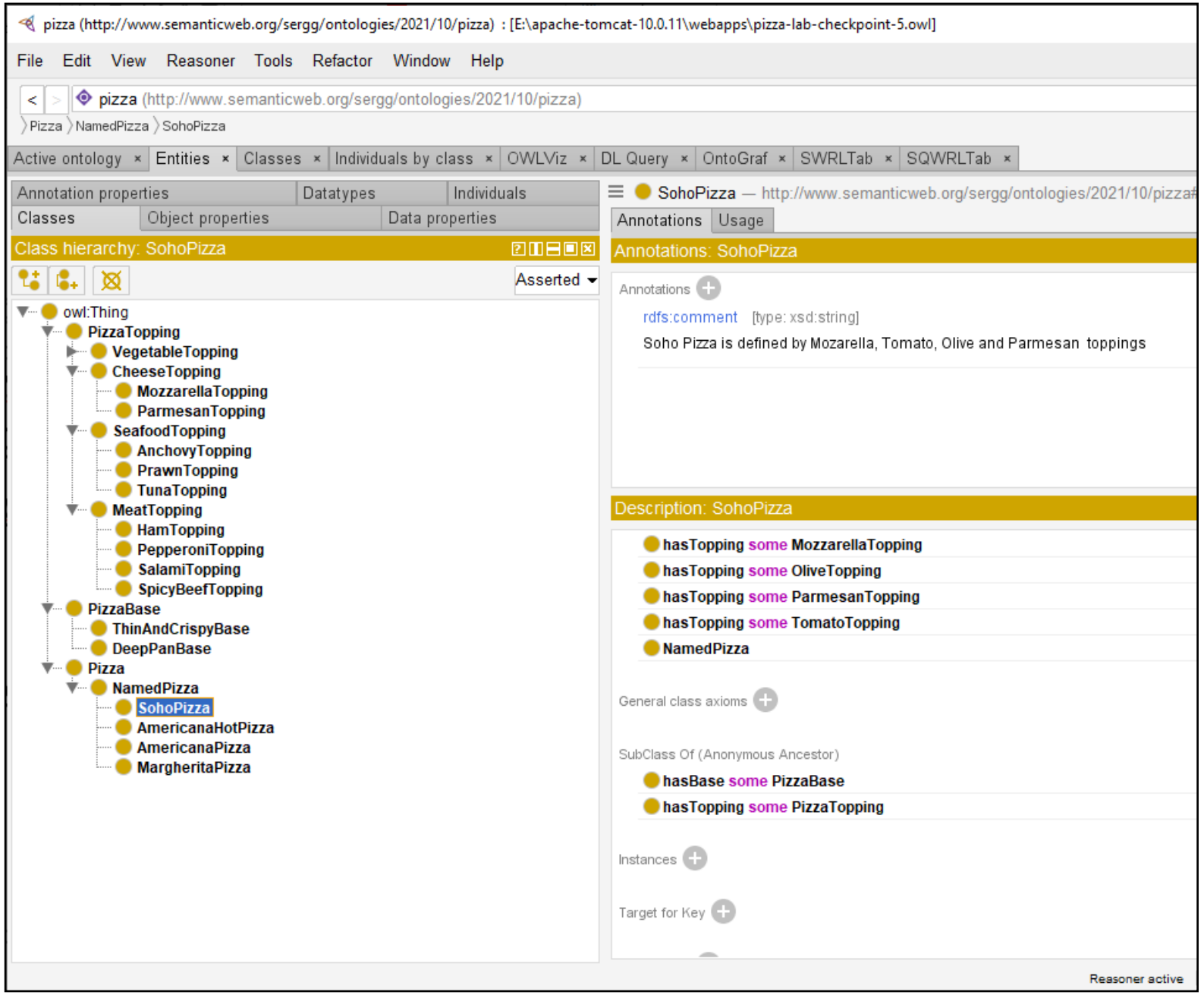

pizza (http://www.semanticweb.org/sergg/ontologies/2021/10/pizza) : [E:\apache-tomcat-10.0.11\webapps\pizza-lab-checkpoint-5.owl]
File Edit View Reasoner Tools Refactor Window Help
pizza (http://www.semanticweb.org/sergg/ontologies/2021/10/pizza)
Search...
Pizza NamedPizza AmericanaPizza
Active ontology Entities Classes Individuals by class OWLViz DL Query OntoGraf SWRLTab SQWRLTab
Annotation properties Datatypes Individuals
Classes Object properties Data properties
Class hierarchy: AmericanaPizza
Asserted
owl:Thing
PizzaTopping
VegetableTopping
CheeseTopping
MozzarellaTopping
ParmesanTopping
SeafoodTopping
AnchovyTopping
PrawnTopping
TunaTopping
MeatTopping
HamTopping
PepperoniTopping
SalamiTopping
SpicyBeefTopping
PizzaBase
ThinAndCrispyBase
DeepPanBase
Pizza
NamedPizza
AmericanaPizza
MargheritaPizza
AmericanaPizza — http://www.semanticweb.org/sergg/ontologies/2021/10/pizza#AmericanaPizza
Annotations Usage
Annotations: AmericanaPizza
Annotations
AmericanaPizza
owl:backwardCompatibleWith
owl:deprecated
owl:incompatibleWith
owl:priorVersion
owl:versionInfo
rdfs:comment
rdfs:isDefinedBy
rdfs:label
rdfs:seeAlso
Literal Entity IRI IRI Editor Property values
AmericanaPizza is defined by Mozarella, Tomato and Pepperoni toppings
Type xsd:string
Lang
Aceptar
Cancelar
Reasoner state out of sync with active ontology
Show Inferences

Crear más clases

- Creamos la clase *AmericanaHotPizza* duplicando *AmericanaPizza* y añadiendo *JalapenoPepperTopping*
- Creamos *SohoPizza* clonando *MargheritaPizza* y añadiendo *OliveTopping* y *ParmesanTopping*

- A continuación, hacer que todas las subclases de NamedPizza sean Disjuntas (a partir de cualquier de ellas, seleccionando las otras 3)




pizza (http://www.semanticweb.org/sergg/ontologies/2021/10/pizza) : [E:\apache-tomcat-10.0.11\webapps\pizza-lab-checkpoint-5.owl]
File Edit View Reasoner Tools Refactor Window Help
pizza (http://www.semanticweb.org/sergg/ontologies/2021/10/pizza)
Pizza NamedPizza SohoPizza
Active ontology Entities Classes Individuals by class OWLViz DL Query OntoGraf SWRLTab SQWRLTab
Annotation properties Datatypes Individuals
Classes Object properties Data properties
Class hierarchy: SohoPizza
Asserted
owl:Thing
PizzaTopping
VegetableTopping
CheeseTopping
MozzarellaTopping
ParmesanTopping
SeafoodTopping
AnchovyTopping
PrawnTopping
TunaTopping
MeatTopping
HamTopping
PepperoniTopping
SalamiTopping
SpicyBeefTopping
PizzaBase
ThinAndCrispyBase
DeepPanBase
Pizza
NamedPizza
SohoPizza
AmericanaHotPizza
AmericanaPizza
MargheritaPizza
SohoPizza — http://www.semanticweb.org/sergg/ontologies/2021/10/pizza#
Annotations Usage
Annotations: SohoPizza
Annotations
rdfs:comment [type: xsd:string]
Soho Pizza is defined by Mozarella, Tomato, Olive and Parmesan toppings
Description: SohoPizza
hasTopping some MozzarellaTopping
hasTopping some OliveTopping
hasTopping some ParmesanTopping
hasTopping some TomatoTopping
NamedPizza
General class axioms
SubClass Of (Anonymous Ancestor)
hasBase some PizzaBase
hasTopping some PizzaTopping
Instances
Target for Key
Reasoner active

Funcionamiento del Razonador

Detectando clases que no pueden tener miembros

- Cuando el razonador detecta que una clase se ha definido de manera que no es posible crear ninguna instancia, el razonador, declara una equivalencia a owl:Nothing.
- Seguramente se debe a un error a la hora de definir la ontología.

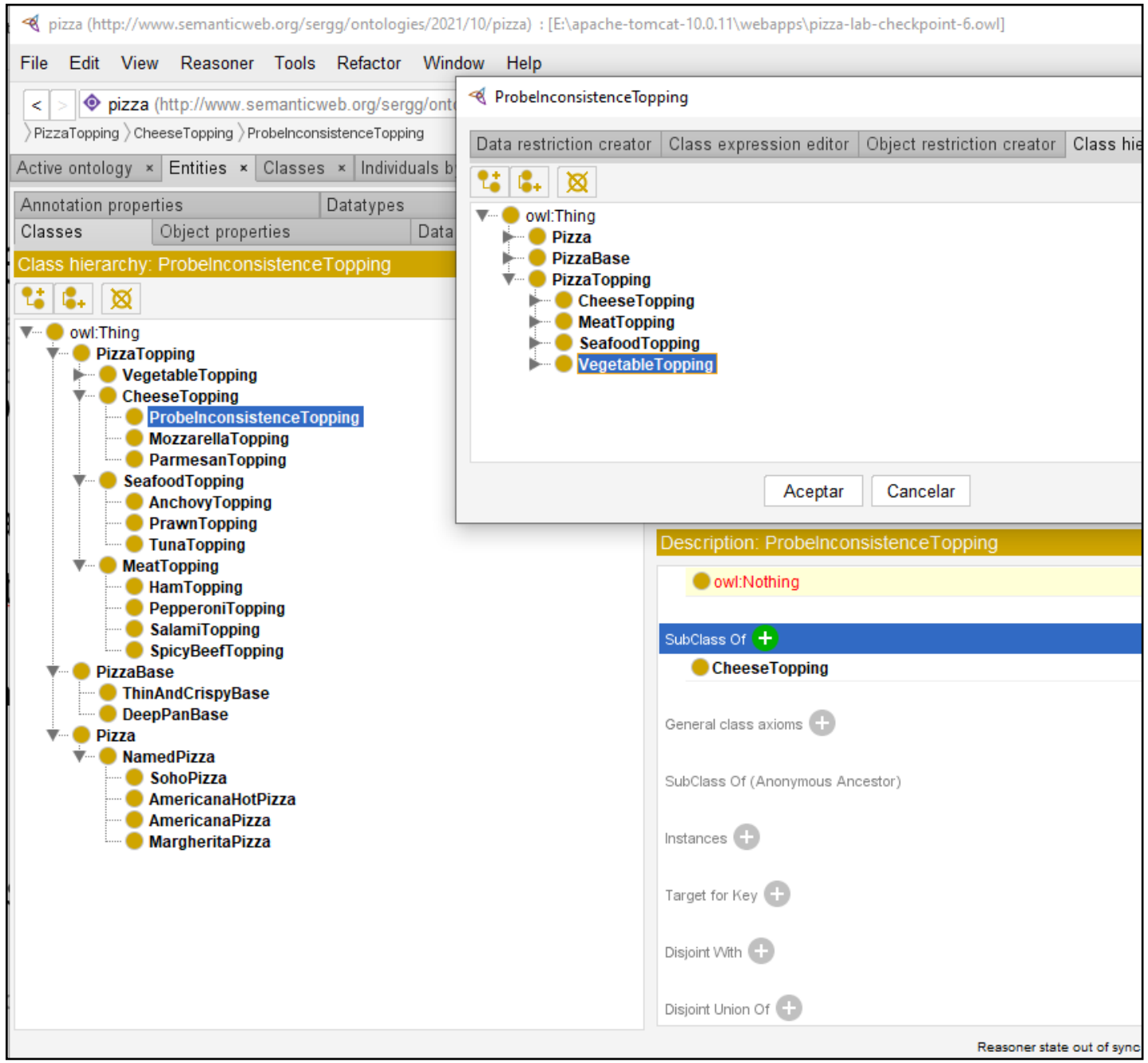
Detección de Inconsistencias

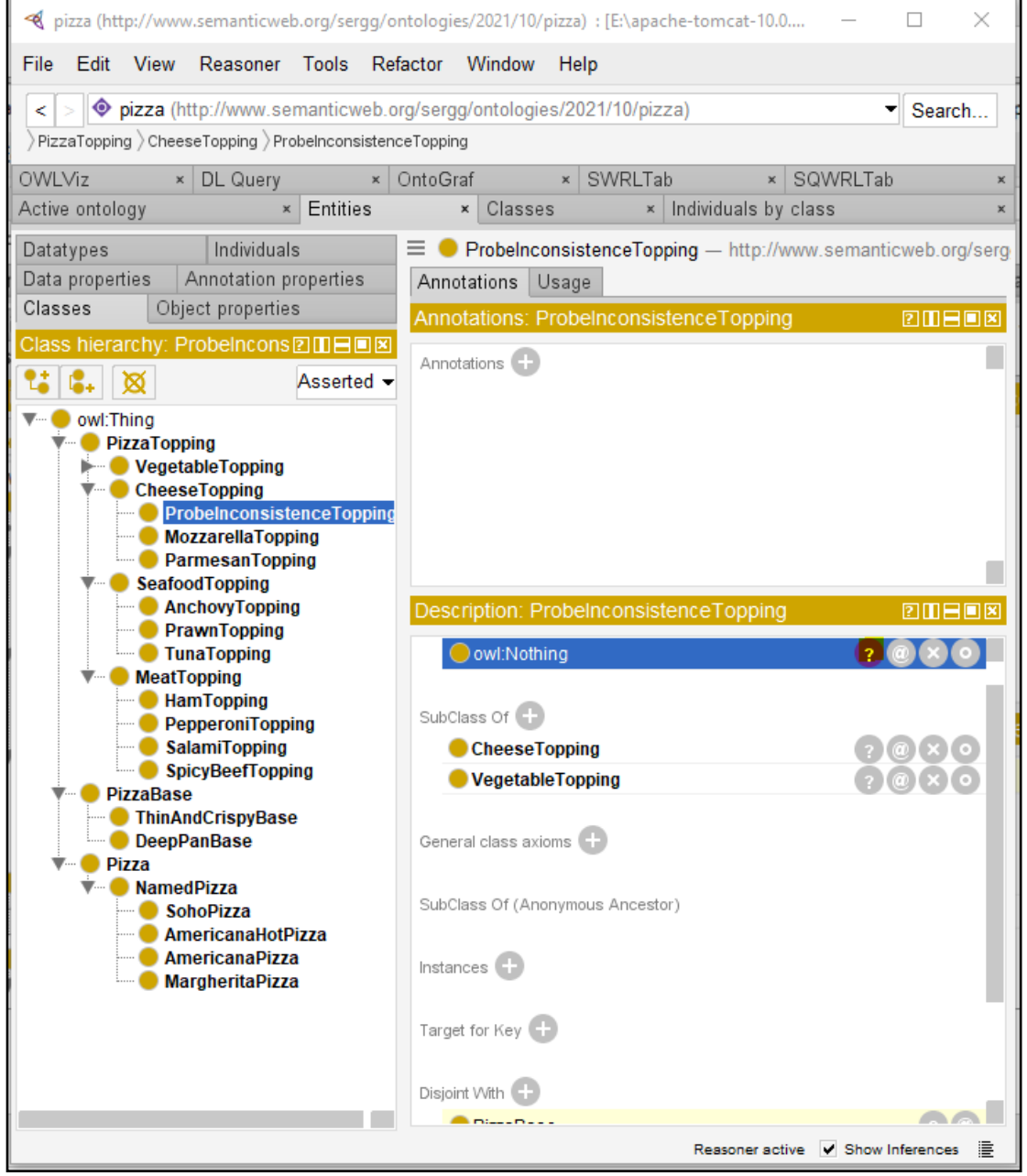
Creamos una clase inconsistente para demostrar el funcionamiento del razonador:

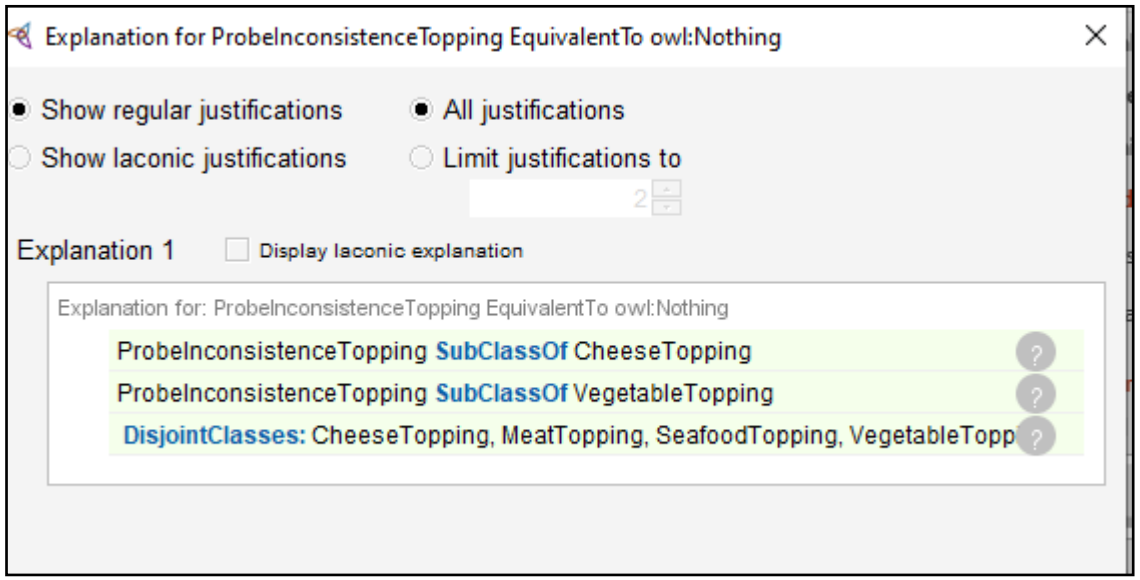
- Desde CheeseTopping crear subclase ProbeInconsistentTopping.
- Seleccionar la clase ProbeInconsistentTopping y añadir una restricción, dando a Subclass of +.
  - Desde Class hierarchy tab seleccionar: VegetableTopping.
- Resincronizar el razonador.



- Mostrar la explicación de por qué se deduce que no puede tener objetos owl:Nothing.


pizza (http://www.semanticweb.org/sergg/ontologies/2021/10/pizza) : [E:\apache-tomcat-10.0.11\webapps\pizza-lab-checkpoint-6.owl]
File Edit View Reasoner Tools Refactor Window Help
pizza (http://www.semanticweb.org/sergg/ont
PizzaTopping CheeseTopping ProbeInconsistenceTopping
Active ontology Entities Classes Individuals b
Annotation properties Datatypes
Classes Object properties Data
Class hierarchy: ProbeInconsistenceTopping
owl:Thing
PizzaTopping
VegetableTopping
CheeseTopping
ProbeInconsistenceTopping
MozzarellaTopping
ParmesanTopping
SeafoodTopping
AnchovyTopping
PrawnTopping
TunaTopping
MeatTopping
HamTopping
PepperoniTopping
SalamiTopping
SpicyBeefTopping
PizzaBase
ThinAndCrispyBase
DeepPanBase
Pizza
NamedPizza
SohoPizza
AmericanaHotPizza
AmericanaPizza
MargheritaPizza
ProbeInconsistenceTopping
Data restriction creator
Class expression editor
Object restriction creator
Class hie
owl:Thing
Pizza
PizzaBase
PizzaTopping
CheeseTopping
MeatTopping
SeafoodTopping
VegetableTopping
Aceptar
Cancelar
Description: ProbeInconsistenceTopping
owl:Nothing
SubClass Of
CheeseTopping
General class axioms
SubClass Of (Anonymous Ancestor)
Instances
Target for Key
Disjoint With
Disjoint Union Of
Reasoner state out of sync

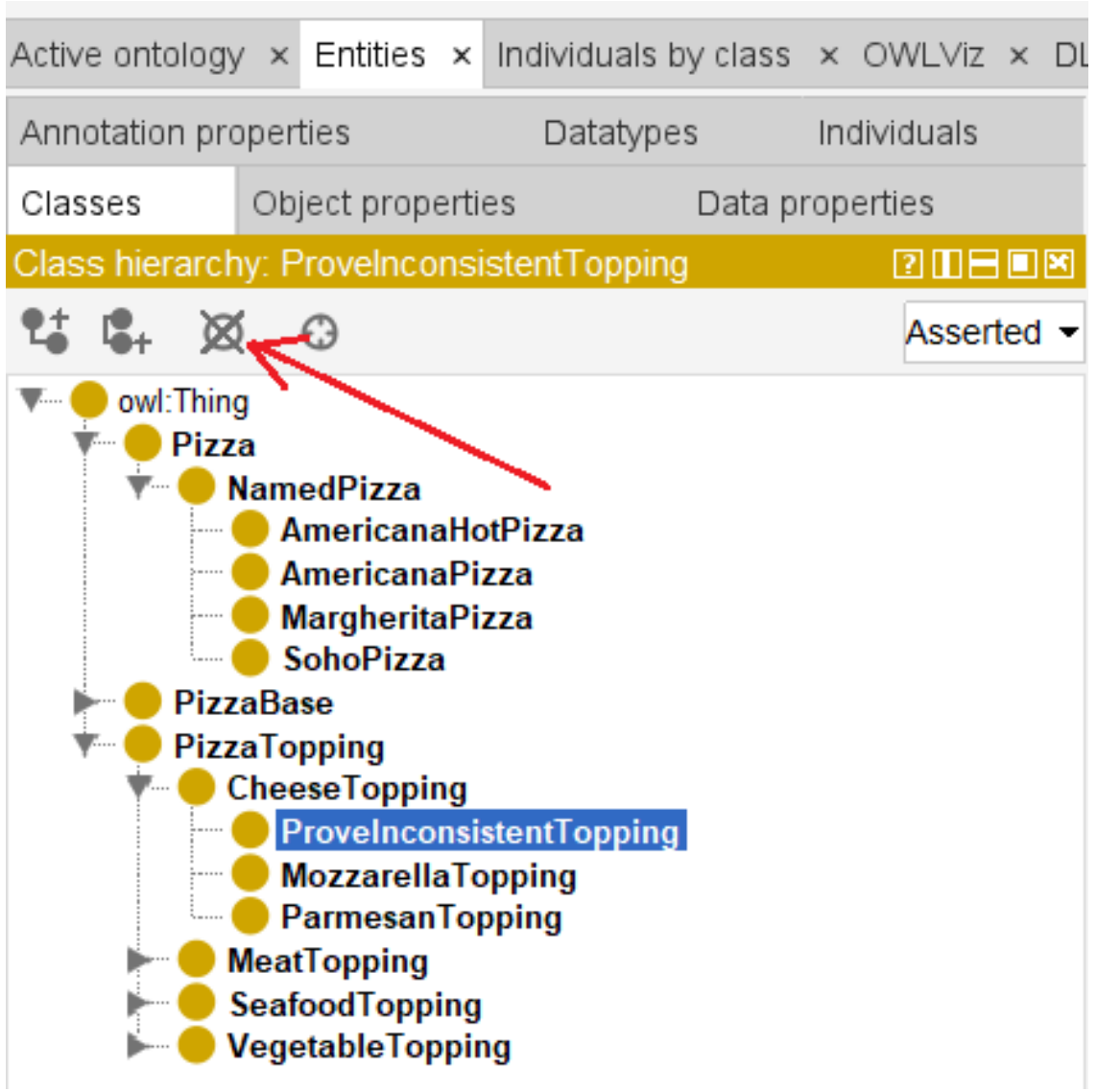




- Cuando el razonador detecta que una clase se ha definido de manera que no es posible crear ninguna instancia, el razonador, declara una equivalencia a owl:Nothing.

Resolver la inconsistencia

- En este ejemplo, basta con eliminar la clase ProbeInconsistentTopping.



Clases Primitivas y Definidas

Describir y Definir Clases

Las Propiedades nos permiten describir las clases:

- Clases **Primitivas**: Proporcionan condiciones necesarias para formar parte de ella (por ejemplo, la clase Pizza), pero no suficientes.
- Clases **Definidas**: Proporcionan condiciones necesarias y suficientes para confirmar que se forma parte de ella.

- Clases **Anónimas**: Son clases auxiliares que genera el razonador para hacer inferencias (por ejemplo, hasBase some PizzaBase)

Clases primitivas

- O simplemente clases.
- Se describen mediante propiedades necesarias.
- Los miembros de la clase tienen todas las propiedades.
- Permiten la herencia de propiedades.

- Clase Pizza: hasBase some PizzaBase, hasTopping some pizzaTopping.
- Todos los individuos de la clase Pizza tienen estas propiedades.
- Tener estas propiedades **no es suficiente** para ser miembro de la clase Pizza.

Clases definidas

- Se describen mediante propiedades necesarias y **suficientes**.
- Los miembros de la clase tienen todas las propiedades.
- Permiten la herencia de propiedades.

- Además: tener estas propiedades **es suficiente** para ser miembro de la clase.

Clasificación y Clases definidas

- Clasificación: asignar una clase a un individuo.

- Las clases definidas permiten la clasificación.
- Por proporcionar condiciones necesarias y **suficientes**.

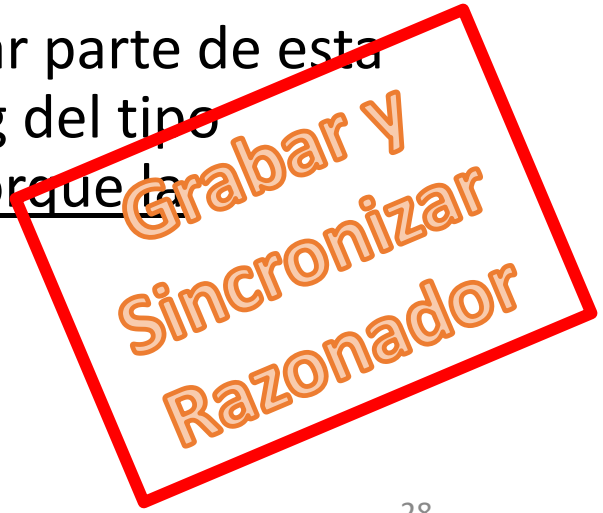
- Las clases primitivas no permiten la clasificación: solo proporcionan condiciones necesarias.

Creación de clases primitivas y definidas

Creamos CheesyPizza (Primitiva):

- Creamos clase CheesyPizza como subclase de Pizza.
- Seleccionando CheesyPizza, añadimos la restricción SubClass Of +
  - hasTopping some CheeseTopping.

Resumen: La descripción de CheesyPizza indica que, para formar parte de esta clase, tiene que ser Pizza y tiene que tener al menos un topping del tipo CheeseTopping (condiciones necesarias, pero no suficientes, porque la definición de la clase no está cerrada: no es una clase definida)



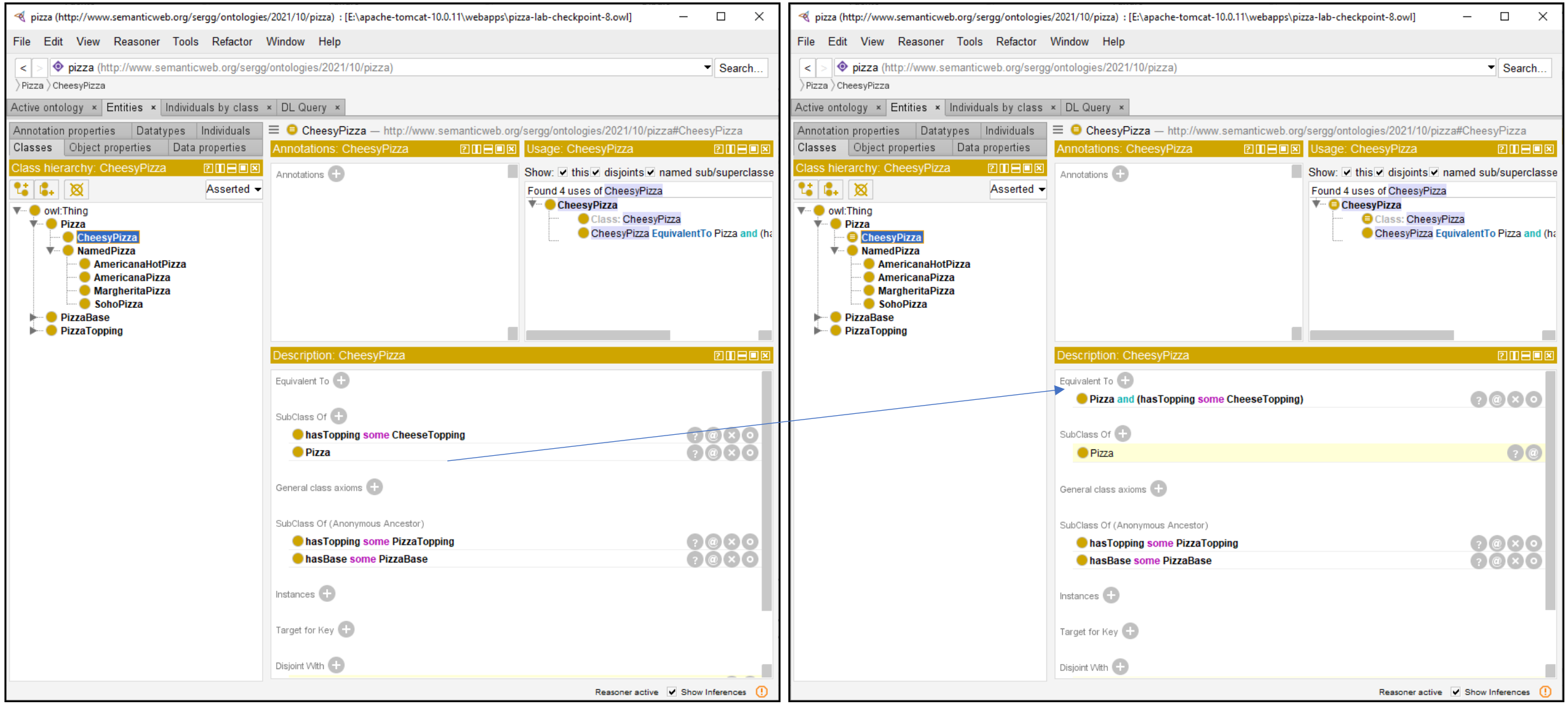
Creación de clases primitivas y definidas

Grabar y Sincronizar Razonador

Convertimos CheesyPizza en clase Definida: (ver página siguiente)

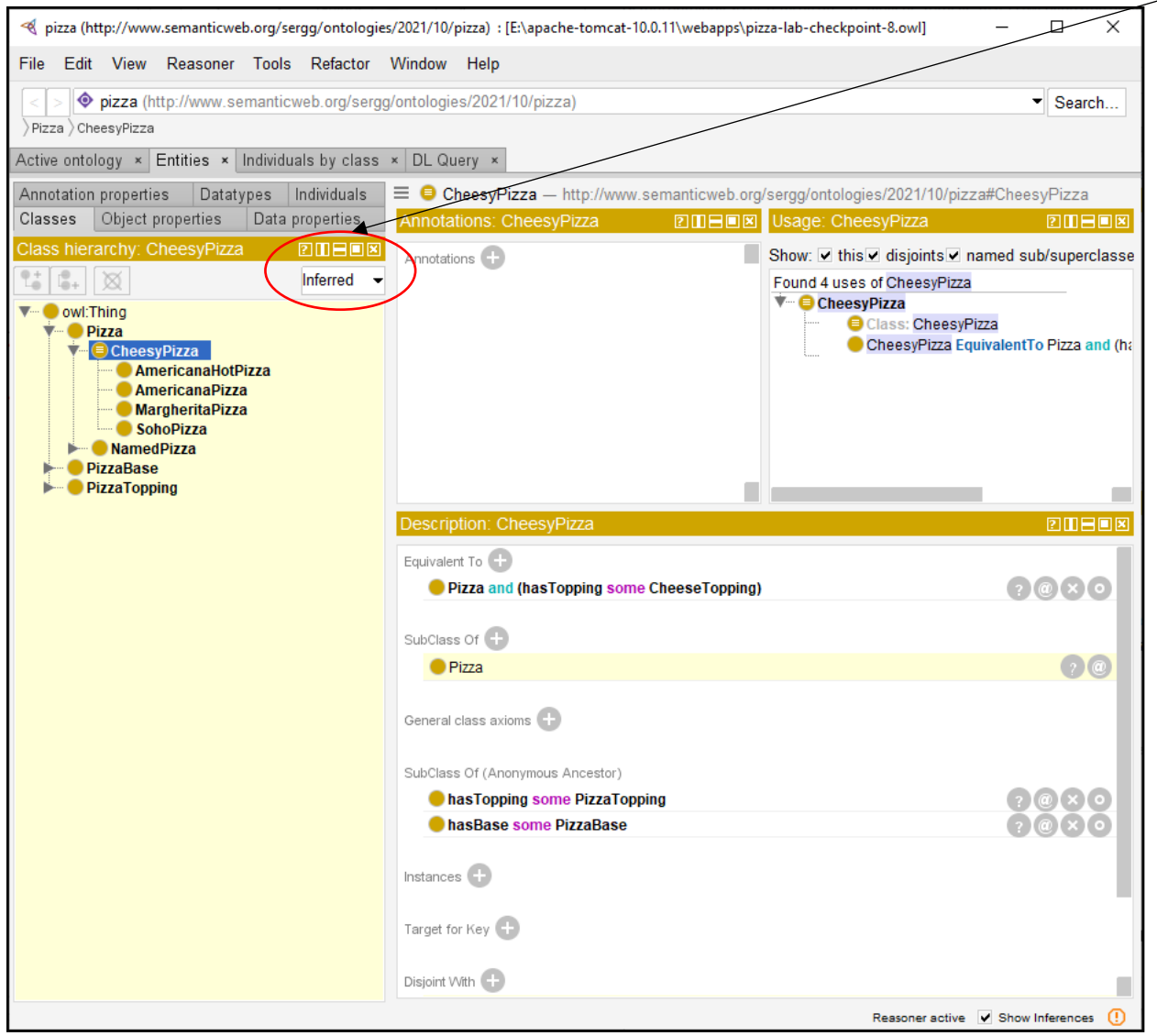
- Seleccionamos CheesyPizza -> Edit -> Convert to defined class
  - A partir de ahora, veremos la clase con 3 líneas en la vista de clases
  - Aparece **“Equivalent To”,** que define las condiciones necesarias y suficientes para formar parte de la clase CheesyPizza
  - Gracias al razonador, en la vista Class Hierarchy (Inferred) veremos que se han categorizado elementos bajo CheesyPizza que no estaban antes (ver dos páginas más adelante)
    - AmericanaHotPizza, AmericanaPizza, MargheritaPizza, SohoPizza (porque tienen topping MozarellaTopping o ParmesanTopping)

Resumen: La descripción de CheesyPizza indica que, para formar parte de esta clase, tiene que ser Pizza y tiene que tener al menos un topping del tipo CheeseTopping, como condiciones necesarias y suficientes. Esto queda fijado en la sección Equivalent To.



Conversión de clase Primitiva a Clase Definida

Cambiar “Asserted” (afirmado) por “Inferred” (inferido)



Restricción Universal (para todo…)

- Hasta ahora, hemos usado restricciones existenciales (*CheesyPizza*, si es *Pizza* y tiene un *CheesyTopping*)
- Ahora usaremos restricciones universales, para crear una clase que incluya únicamente aquellas pizzas que tienen ingredientes vegetarianos, y solo vegetarianos.

Creamos una Clase usando restricción Universal

- Seleccionamos Pizza y creamos una subclase: VegetarianPizza
- Seleccionamos VegetarianPizza y restringimos con Subclass Of +
- Class expression editor…. ¿Qué axioma *de lógica descriptiva usaremos?*

    - *PIENSA UNA SOLUCION ANTES DE IR A LA SIGUIENTE PÁGINA*

- Dato: Las clases que contienen ingredientes no vegetarianos son las que tienen topping de queso o vegetal.
- Pista: Debe usarse “only”

Creamos una Clase usando restricción Universal

- Seleccionamos la clase Pizza y creamos una subclase: VegetarianPizza
- Seleccionamos VegetarianPizza y restringimos con Subclass Of +
- Class expression editor…. ¿Qué axioma de lógica descriptiva usaremos?

  - hasTopping only (VegetableTopping or CheeseTopping)

Grabar y Sincronizar Razonador

- **Convertimos** la clase VegetarianPizza en clase **Definida**.
  - Para que nos permita inferir qué Pizzas son vegetarianas (clasificación).

Comprobamos el resultado...

- Examinar la jerarquía de clases inferida.
- Comprobar que VegetarianPizza no tiene subclases.

- Este resultado nos puede sorprender, pues VegetarianPizza es una clase definida y debería inferir que pizzas son vegetarianas:
  - ¿Dónde deberían estar las clases MargheritaPizza y SohoPizza)?
  - ¿Por qué no están clasificadas como VegetarianPizza?

Comprobamos el resultado...

- Examinar la jerarquía de clases inferida.
- Comprobar que VegetarianPizza no tiene subclases

- Este resultado nos puede sorprender, pues VegetarianPizza es una clase definida y debería inferir que pizzas son vegetarianas:
  - ¿Dónde deberían estar las clases MargheritaPizza y SohoPizza)?
  - ¿Por qué no están clasificadas como VegetarianPizza?

- Porque, de acuerdo con su descripción, pueden tener otros Toppings.
- OWL no utiliza la suposición de mundo cerrado (CWA).

OWA vs CWA

CWA: Suposición del Mundo Cerrado (Close World Assumptiom)

- CWA: todo lo que no es explícitamente cierto, es falso.
- CWA es de utilidad cuando nuestro conocimiento es completo.

OWA: Suposición del Mundo Abierto (Open World Assumptiom)

- OWA: solo es cierto lo que está explícitamente representado, (o se puede derivar con reglas de inferencia sólidas en sistemas de inferencia)
  - Recordar: SLDNF no es solida.
- OWA es de utilidad cuando nuestro conocimiento es incompleto.

OWA, CWA y clases

- OWA (Open World Assumption) – Suposición de mundo abierto
  - Si no se ha definido alguna información, ésta puede existir, es decir que la definición proporcionada permanece abierta.
  - OWL considera que la información puede no estar integrada aún en nuestra ontología.
- CWA (Closed World Assumption)- Suposición de mundo cerrado
  - Si no se ha definido alguna información, ésta no existe, es decir, que la definición proporcionada es completa.
  - ¡¡Más común en los lenguajes de programación y en lenguajes de representación del conocimiento!!

El razonador supone que las pizzas Margherita y Soho podrían tener otros toppings no vegetarianos, dado que no se ha definido que únicamente pertenezcan a esas clases los individuos que tengan esos toppings, sino que son necesarios para formar parte de la clase, aunque podrían tener más, de manera que quedarían fuera de la clase Vegetariana y seguirían siendo Margherita o Soho.

Conocimiento completo y OWA

- Algunas partes de nuestro conocimiento son completas: OWA no es de utilidad en estos casos.
- Por ejemplo, *MargheritaPizza* solo puede tener como Topping mozzarela y tomate.
- Ya hemos especificado que puede tener mozzarella y tomate.
- Podemos añadir **axiomas de cierre locales** para que solo puede tener estos Topping

hasTopping only (*MozzarellaTopping* or *TomatoTopping*)



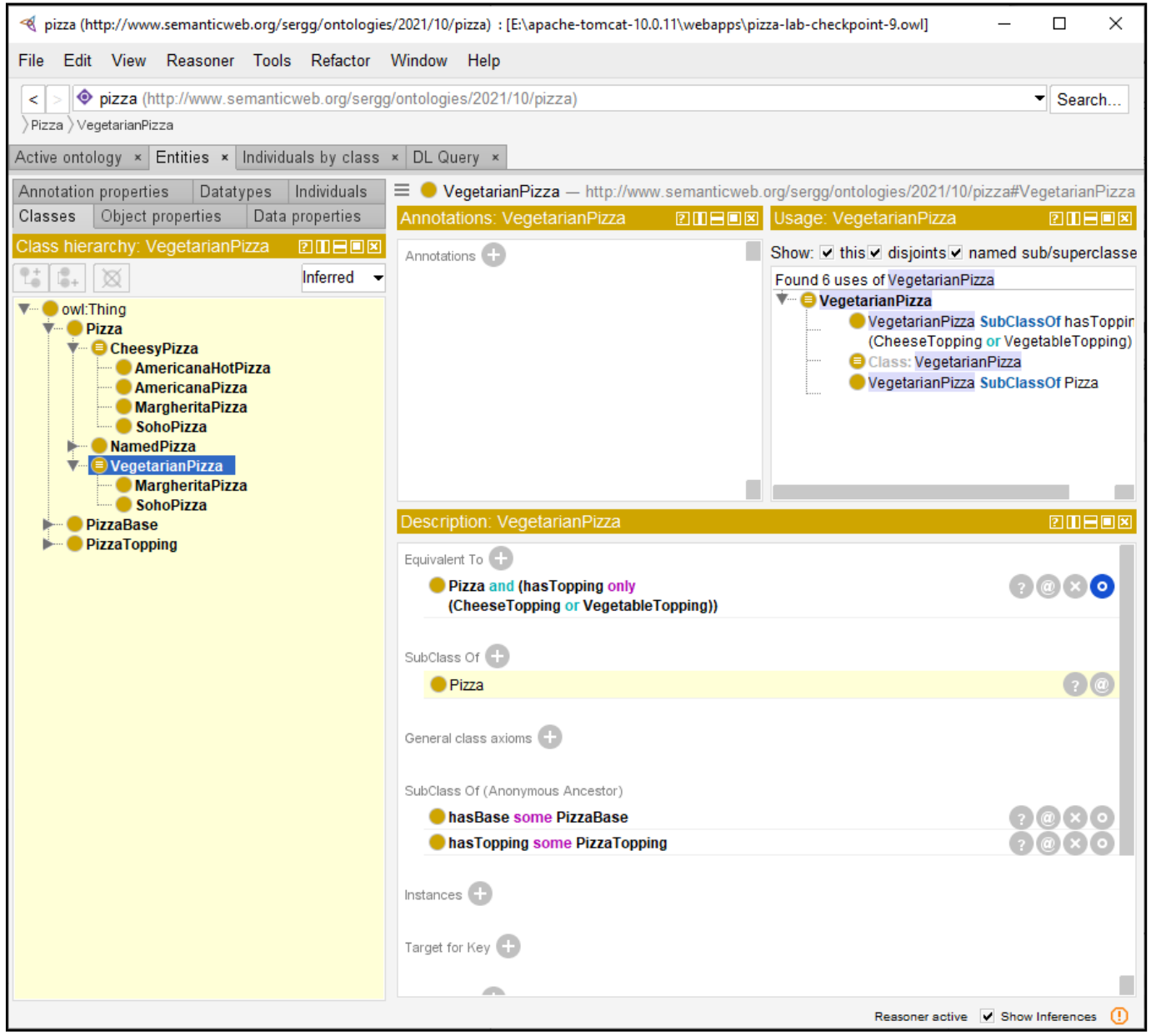
Axioma de Cierre de local

- Si queremos cerrar la información de una clase en una ontología, es necesario añadir el axioma de cierre, que evita que se añadan ingredientes a *MargheritaPizza* y a *SohoPizza*:
  - Seleccionar *MargheritaPizza* => SubClass of +
  - Class expression editor: hasTopping only (*MozzarellaTopping* or *TomatoTopping*)

  - Seleccionar *SohoPizza* => SubClass of +
  - Class expression editor: *hasTopping* only (*MozzarellaTopping or TomatoTopping* or *ParmesanTopping* or *OliveTopping*)

  - Comprobar la jerarquía de clases inferidas.
  - Comprobar las deducciones haciendo click en la **?**





Esta vez sí que se deduce que MargheritaPizza y SohoPizza son subclase de VegetarianPizza, porque tras definir la cláusula de cierre, se cumplen las condiciones necesarias y suficientes para pertenecer a la clase.

Atención

- Los axiomas de cierre no convierten a la clase en definida:



Clases Enumeradas

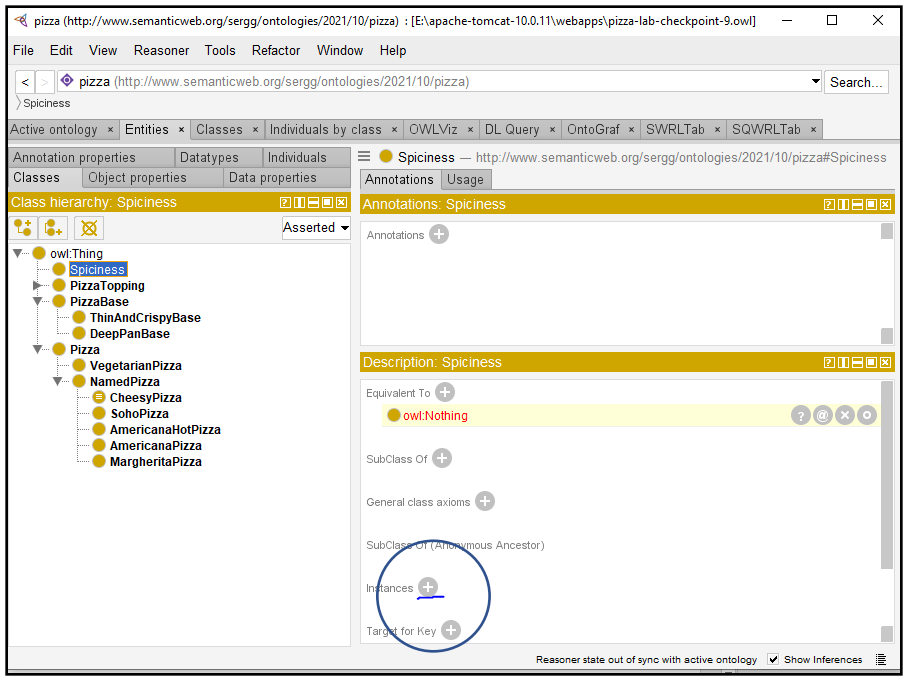
Clases Enumeradas

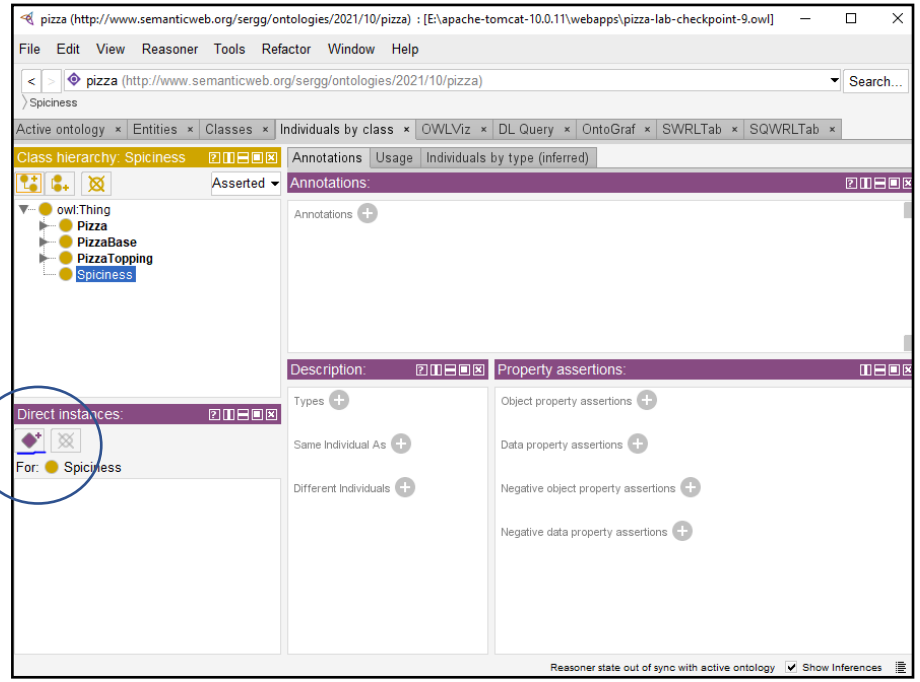
- En ocasiones, se necesita incluir en una Ontología información específica sobre clases, como, por ejemplo, una propiedad que pueda tener un conjunto de valores finito.

- Para crearlas, crearemos una clase enumerada, y crearemos individuos para cada uno de los valores posibles.

Creamos una clase enumerada para el nivel de Picante

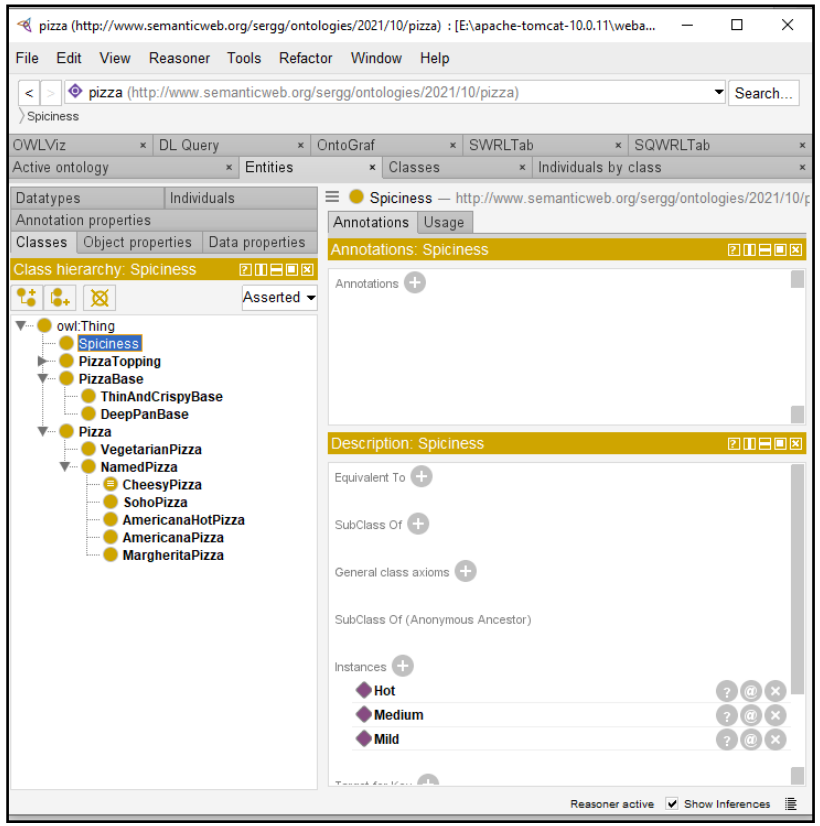
- Seleccionando owl:Thing desde la vista de clases (Asserted), creamos la clase Spiciness.
- A continuación, seleccionamos la clase Spiciness y creamos 3 instancias (hay varias maneras de hacerlo. Se recomienda la segunda: mediante la ventana Direct instances)





Creación de Instancias

- Creamos las instancias (Individuos): Hot, Medium, Mild de clase Spiciness

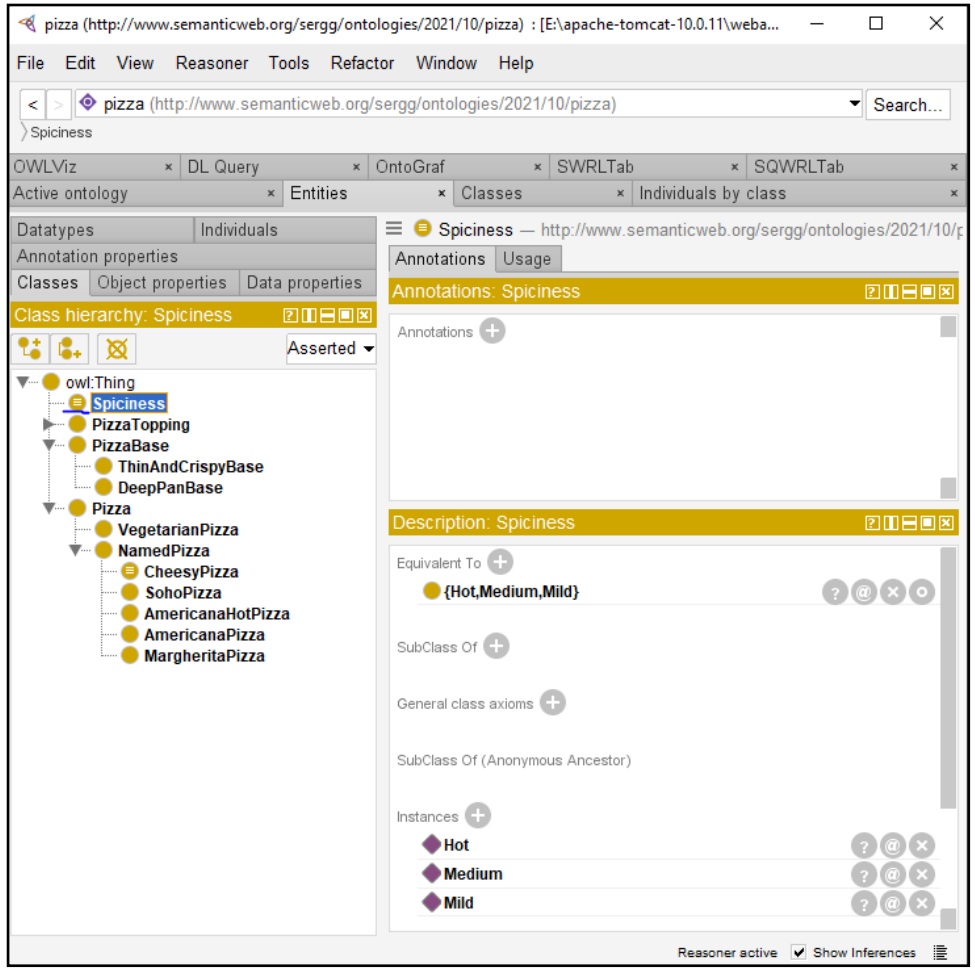


Checkpoint 9

Graber y Sincronizar Razonador

Limitamos los casos posibles de Spiciness

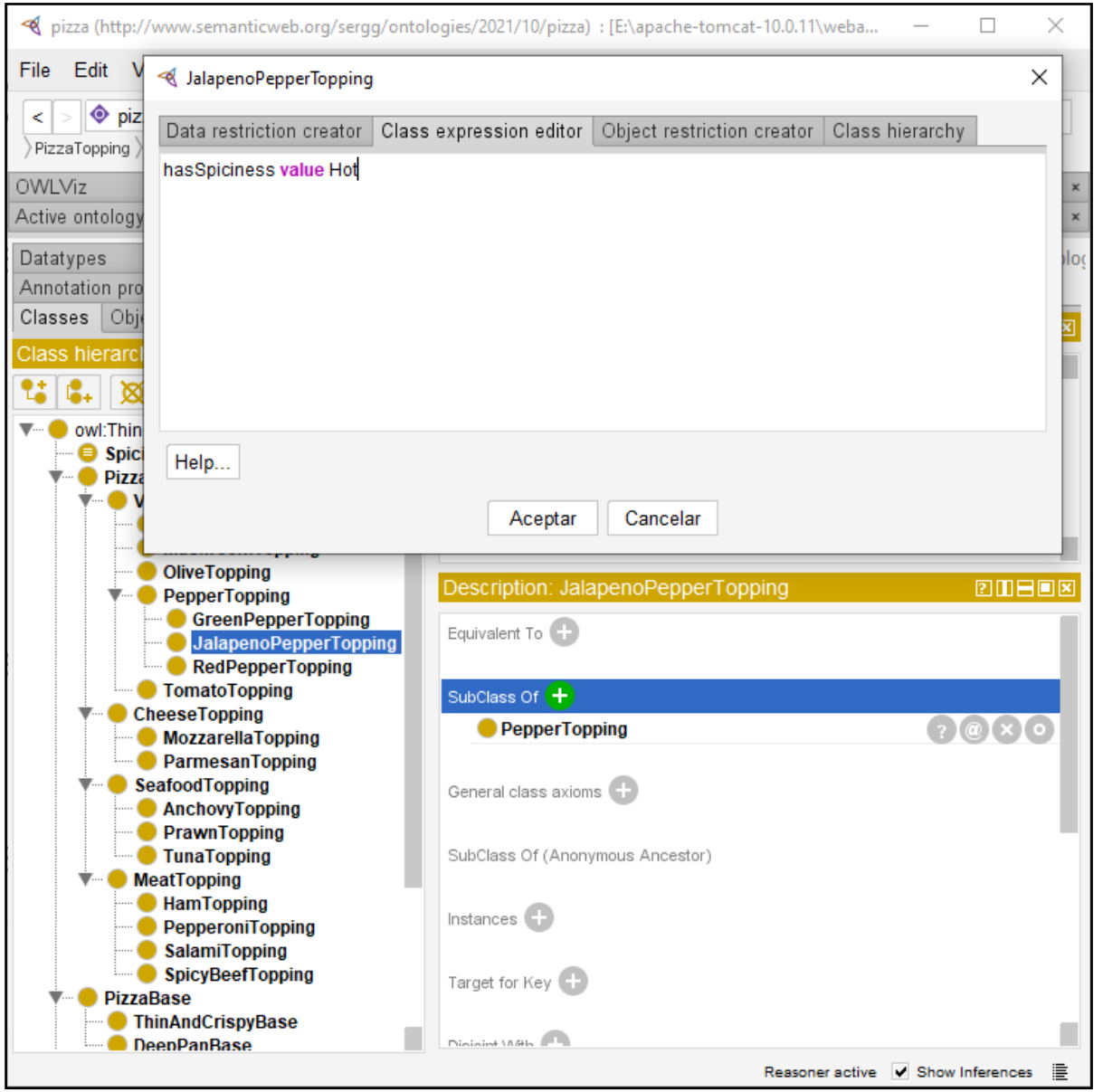
- Una vez creados, seleccionar la clase Spiciness y hacer click en Equivalent To + en la vista de Descripción.
- Seleccionar, Class Expression Editor y añadir el axioma de lógica descriptiva {Hot, Medium, Mild}
- Ejecutar el razonador => Ahora, la clase es Definida (debido a que tiene la sección *Equivalent To* definida), lo sabemos gracias a las 3 líneas horizontales



Asociar un nivel de Picante a las clases

- Seleccionar Object Properties y crear una nueva propiedad llamada: **hasSpiciness** que sea subpropiedad, como las anteriores propiedades de TopObjectProperty.
- Dominio: PizzaTopping.
- Rango: Spiciness.
- Sincronizar el razonador.

- Desde la vista de clases, seleccionar JalapenoPepperTopping y seleccionar SubClass Of + .
- Indicar hasSpiciness **value** Hot .
- Atención: La restricción se refiere a un individuo en concreto (Hot).



Checkpoint 10

Graber y Sincronizar Reasoner

Crear una clase que represente a las clases picantes (I)

- Seleccionamos la clase Pizza y creamos la subclase SpicyPizza.
- Seleccionamos SpicyPizza y hacemos click en SubClass Of +.

  - ¿Qué axioma habrá que definir para identificar a las clases que son picantes?

  - Pista: tiene que tener al menos un topping que sea picante (Hot).

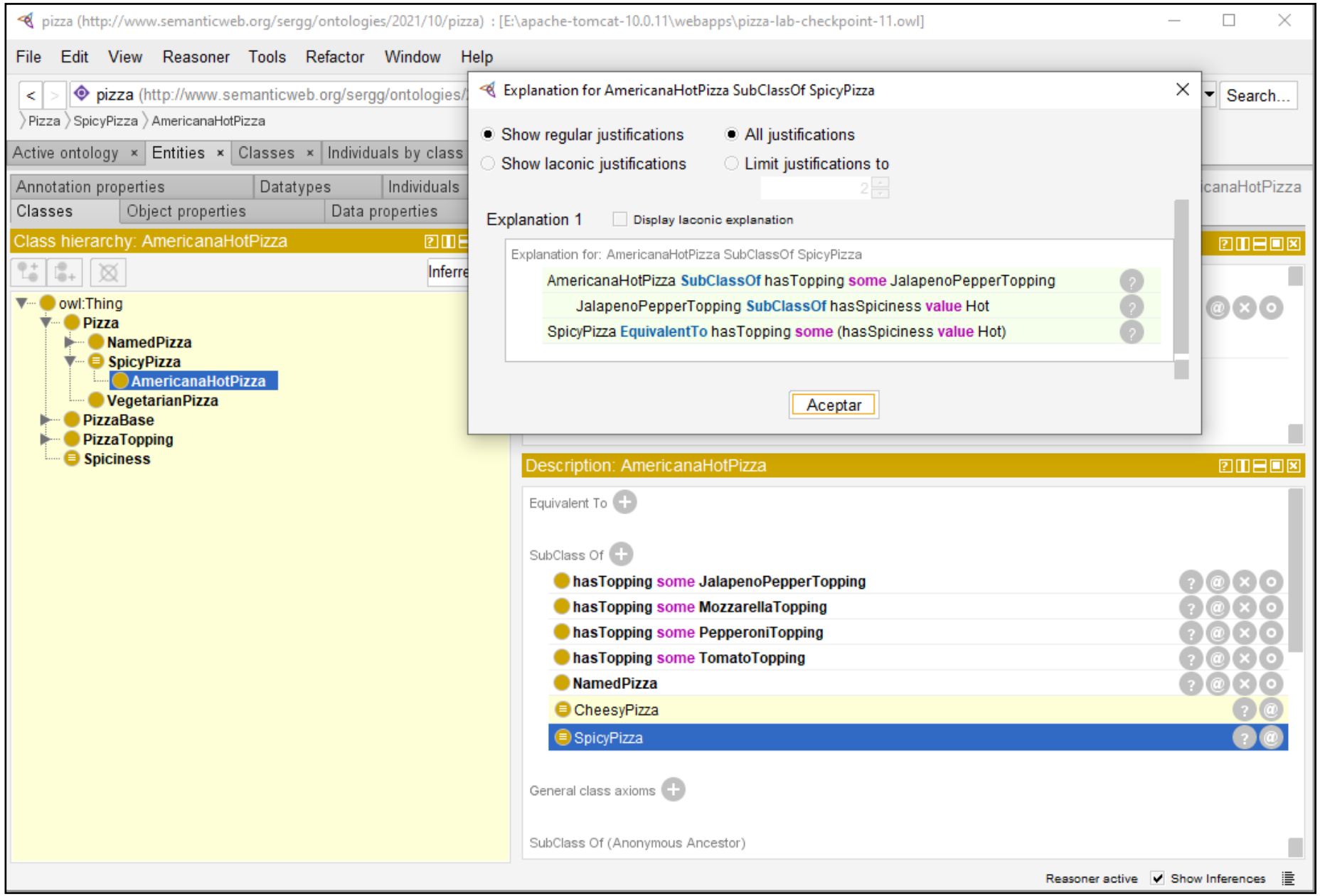
Crear una clase que represente a las clases picantes (II)

- Seleccionamos la clase Pizza y creamos la subclase SpicyPizza.
- Seleccionamos SpicyPizza y hacemos click en SubClass Of +.
  - ¿Qué axioma habrá que definir para identificar a las clases que son picantes?
  - hasTopping some (hasSpiciness value Hot).
- Ejecutar el razonador.
- Observar la vista de Class hierarchy (inferred).
- ¿Se ha añadido la pizza con ingredientes picantes? ¿Por qué?

Crear una clase que represente a las clases picantes (III)

- Ejecutar el razonador.
- Observar la vista de Class hierarchy (inferred)
  - ¿Se ha añadido la pizza con ingredientes picante? ¿Por qué?
- Falta convertirla en Clase Definida para aplicar el cierre de clase.

- Seleccionamos SpicyPizza y la convertimos en clase definida.
- Ejecutar el razonador.
- Observar la vista de Class hierarchy (inferred).



Checkpoint 11

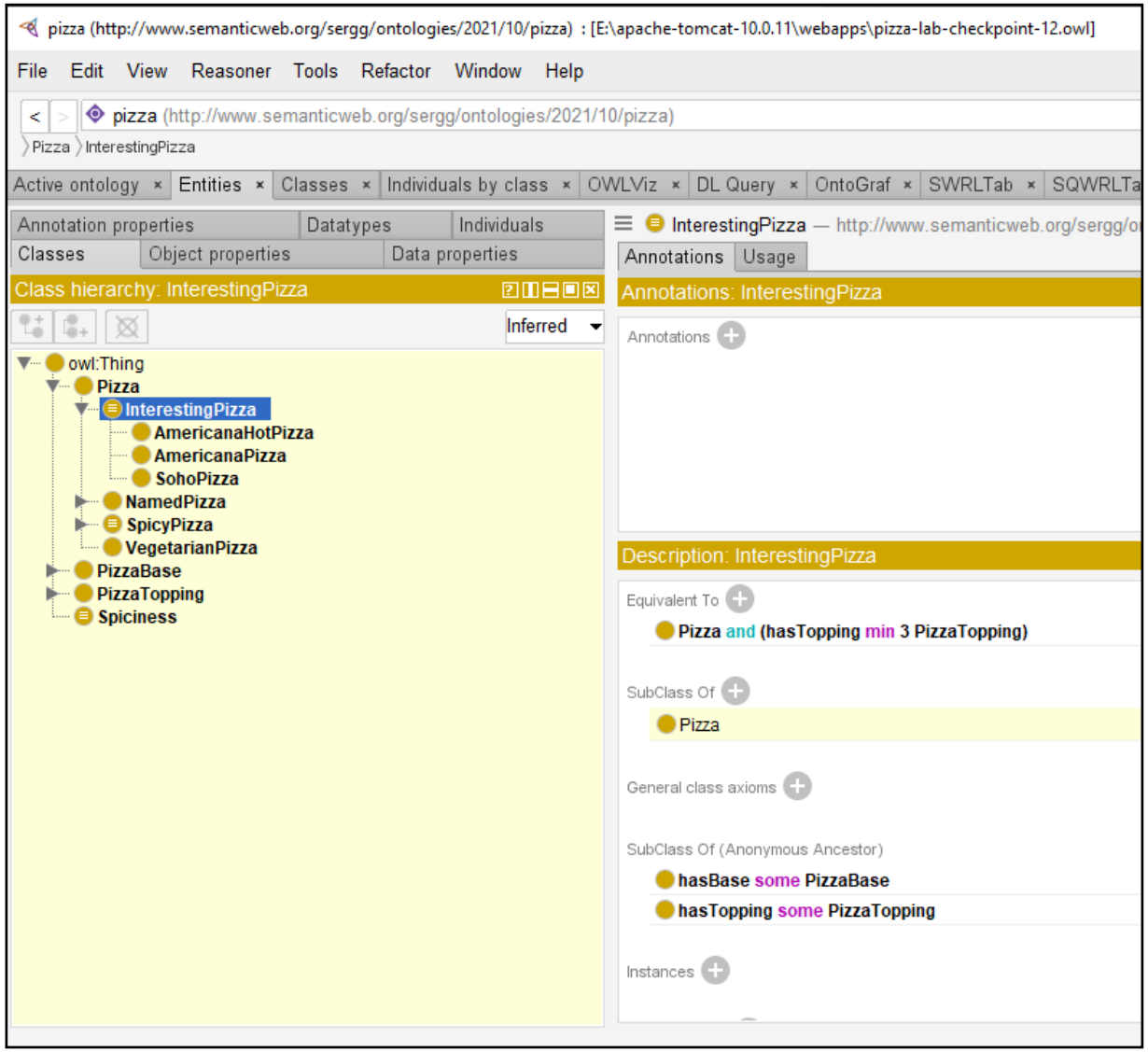
Graber y Sincronizar Reasoner

Mas restricciones sobre propiedades y datos

Restricciones de Cardinalidad

- Sirven para definir una clase en función del número de relaciones con otros individuos o tipos de datos.
- Creamos una clase que represente a las pizzas con al menos 3 ingredientes.

- Seleccionamos la clase Pizza y creamos una subclase InterestingPizza
- Restringimos la clase, haciendo click en SubClass Of +, y en el Class expression editor, introducimos:
  - hasTopping min 3 PizzaTopping
- Convertimos la clase InterestingPizza en clase Definida
- Ejecutamos el razonador
- Comprobamos las clases inferidas en la vista de Class hierarchy (inferred)



Checkpoint 12

Graber y Sincronizar Razonador

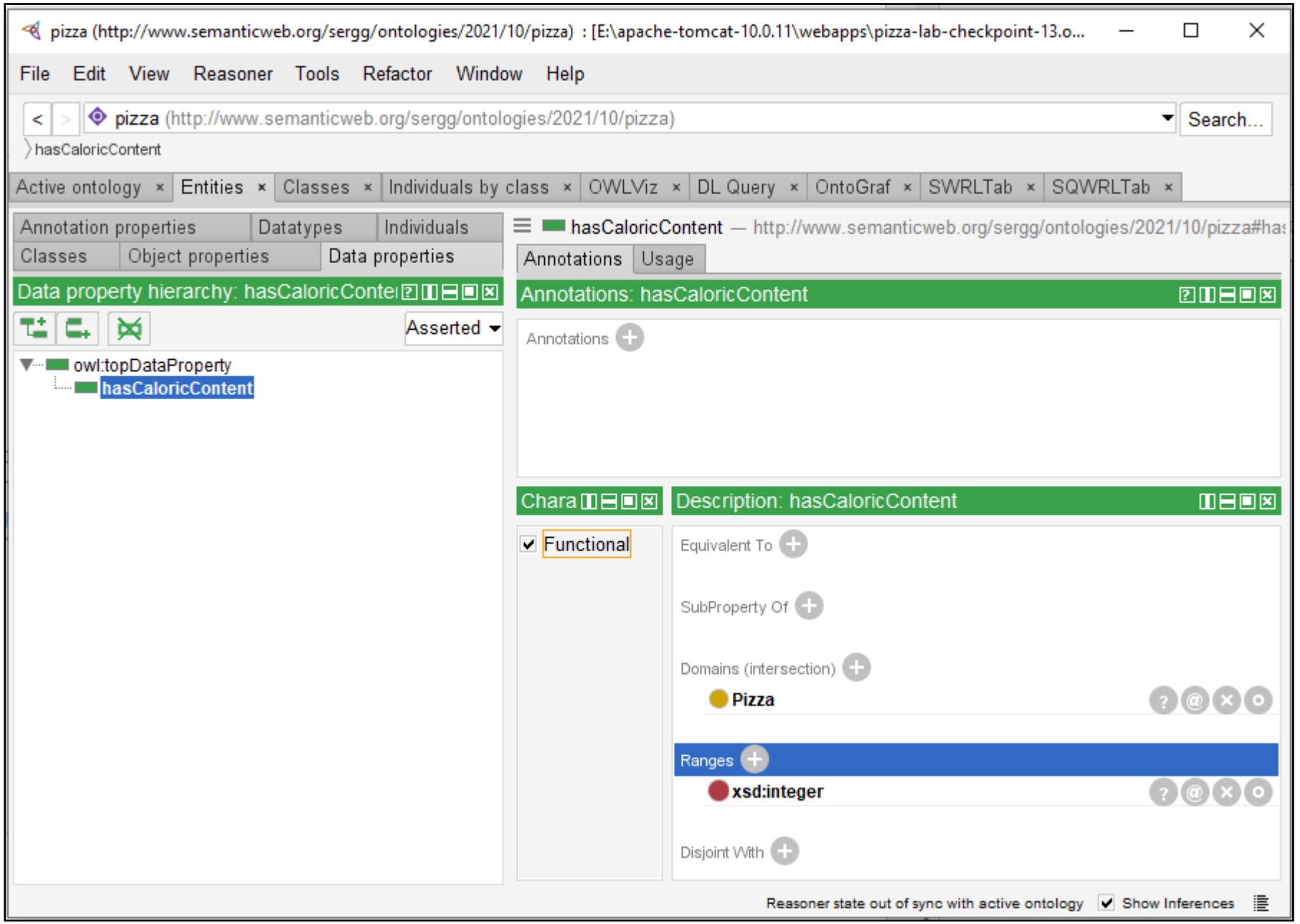
Propiedades de tipos de datos

- OWL se beneficia de una gran librería de tipos de datos predefinidos, generalmente de XML, tales como xsd:string y xsd:integer.

- Ejercicio: Definiremos una propiedad de datos, que nos permita disponer de la información calórica de los individuos, y de identificar, en una clase, las pizzas con alto nivel calórico.

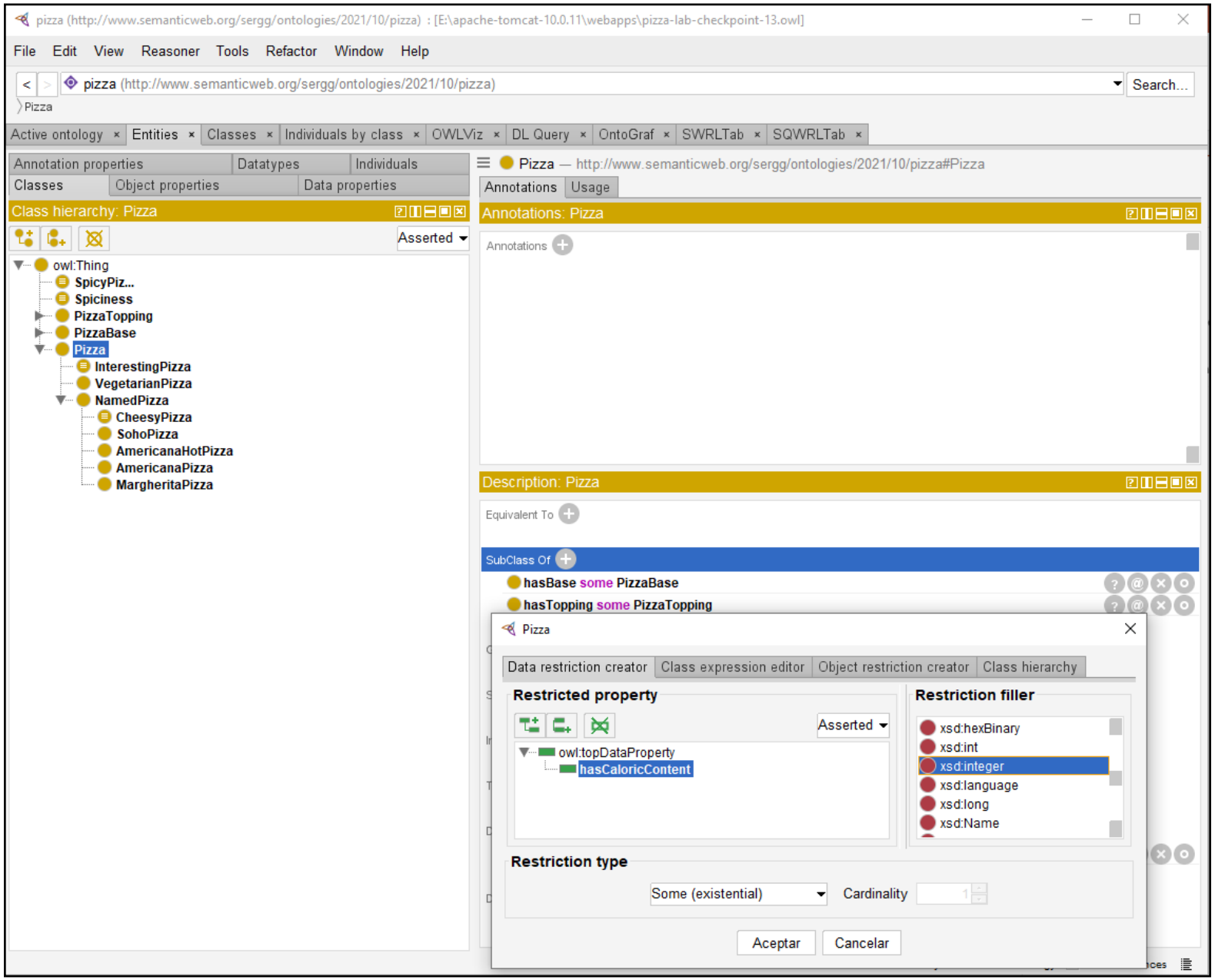
Creamos una propiedad de datos

- Abrimos la vista Window > Tabs > Data properties (ver siguiente transparencia)
- Seleccionamos owl:topDataProperty y creamos una subpropiedad llamada hasCaloricContent (dejar por defecto la opción de no hacer las propiedades disjuntas)
- Dominio: Pizza
- Rango: Built-in ➔ xsd:integer
- Seleccionamos Functional, porque una pizza únicamente puede tener un valor calórico.


pizza (http://www.semanticweb.org/sergg/ontologies/2021/10/pizza) : [E:\apache-tomcat-10.0.11\webapps\pizza-lab-checkpoint-13.o...
File Edit View Reasoner Tools Refactor Window Help
pizza (http://www.semanticweb.org/sergg/ontologies/2021/10/pizza)
Search...
hasCaloricContent
Active ontology Entities Classes Individuals by class OWLViz DL Query OntoGraf SWRLTab SQWRLTab
Annotation properties Datatypes Individuals
Classes Object properties Data properties
Data property hierarchy: hasCaloricConte
Asserted
owl:topDataProperty
hasCaloricContent
hasCaloricContent — http://www.semanticweb.org/sergg/ontologies/2021/10/pizza#has
Annotations Usage
Annotations: hasCaloricContent
Annotations
Chara
Functional
Description: hasCaloricContent
Equivalent To
SubProperty Of
Domains (intersection)
Pizza
Ranges
xsd:integer
Disjoint With
Reasoner state out of sync with active ontology
Show Inferences

Creamos una restricción que da la opción de proporcionar el valor calórico de las pizzas

- Seleccionamos Pizza en la vista de Classes
- Subclass Of +
- Usaremos esta vez: Data Restriction tab
  - seleccionar hasCaloricContent en la vista de Restricted property view
  - Seleccionamos xsd:integer en la vista Restriction filler
  - Tipo de restricción: some (también podría ser exactly 1, pero al haber definido hasCaloricContent como funcional, no es necesario)



Checkpoint 13



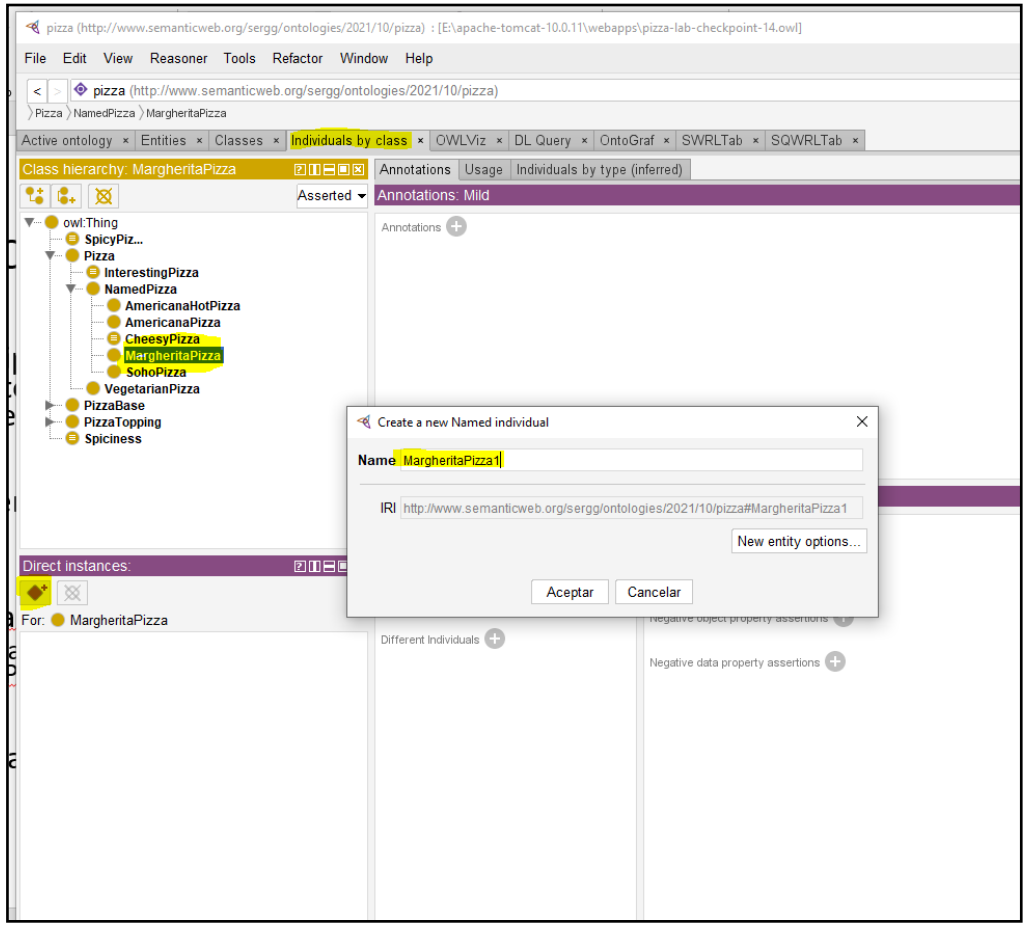
Poblar la base de conocimiento

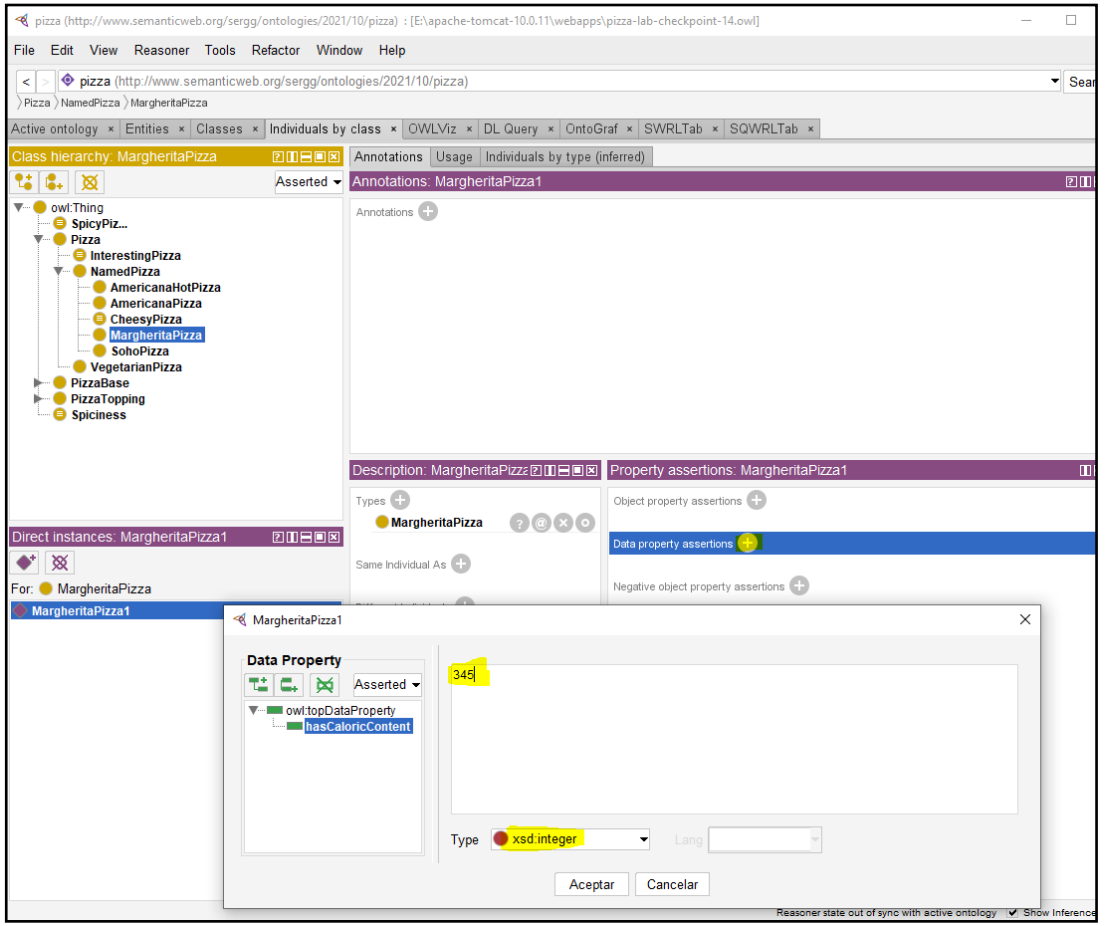
- Creamos varios individuos de tipo Pizza, y especificamos su valor calórico mediante esa propiedad de dato. Introducir 5 instancias con calorías por debajo de 400 y otros 5 por encima de 400 (ver las siguientes dos transparencias)

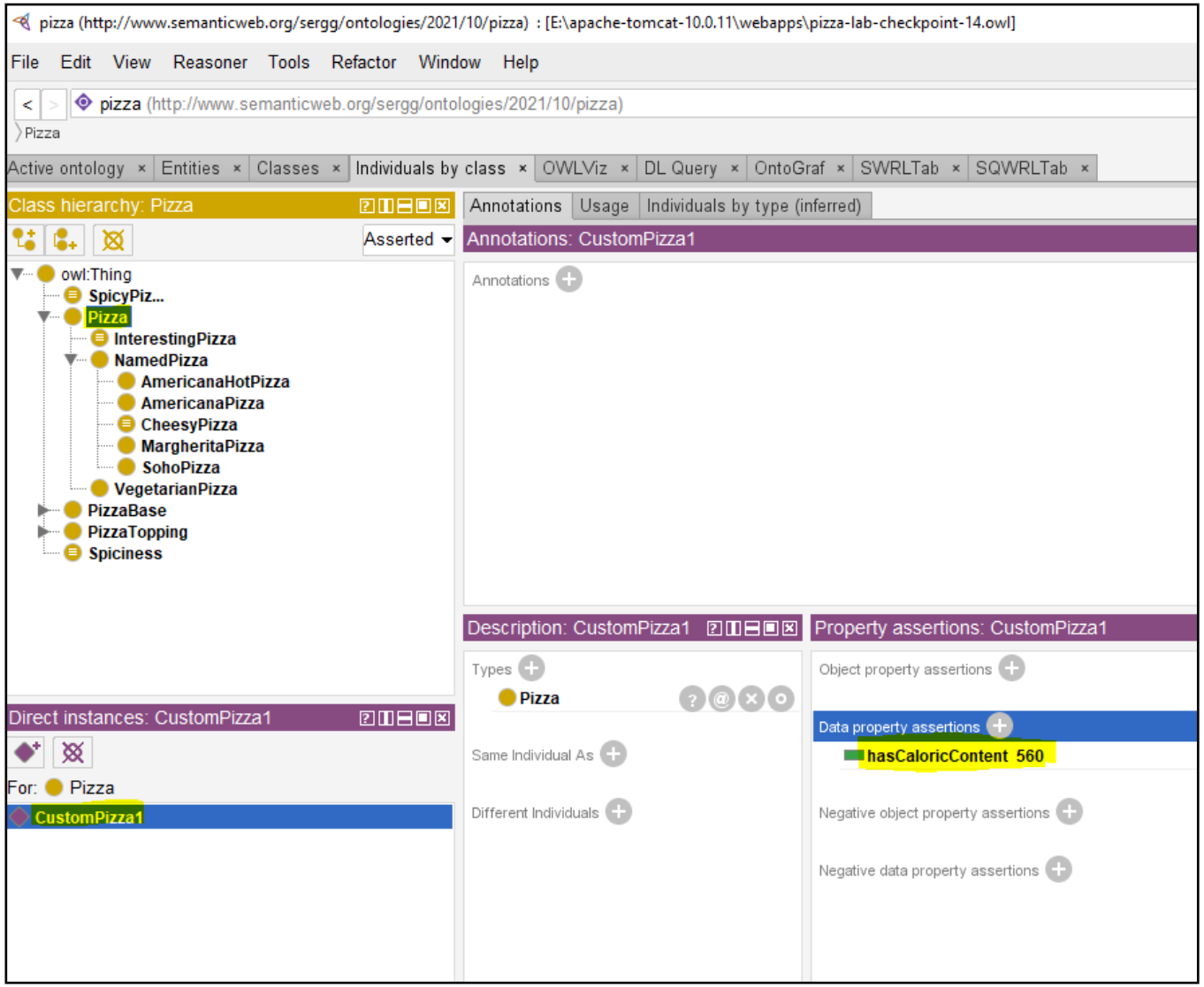
- Convención de nombres: Usaremos CamelBack añadiendo un número. Ejemplo: MargheritaPizza1

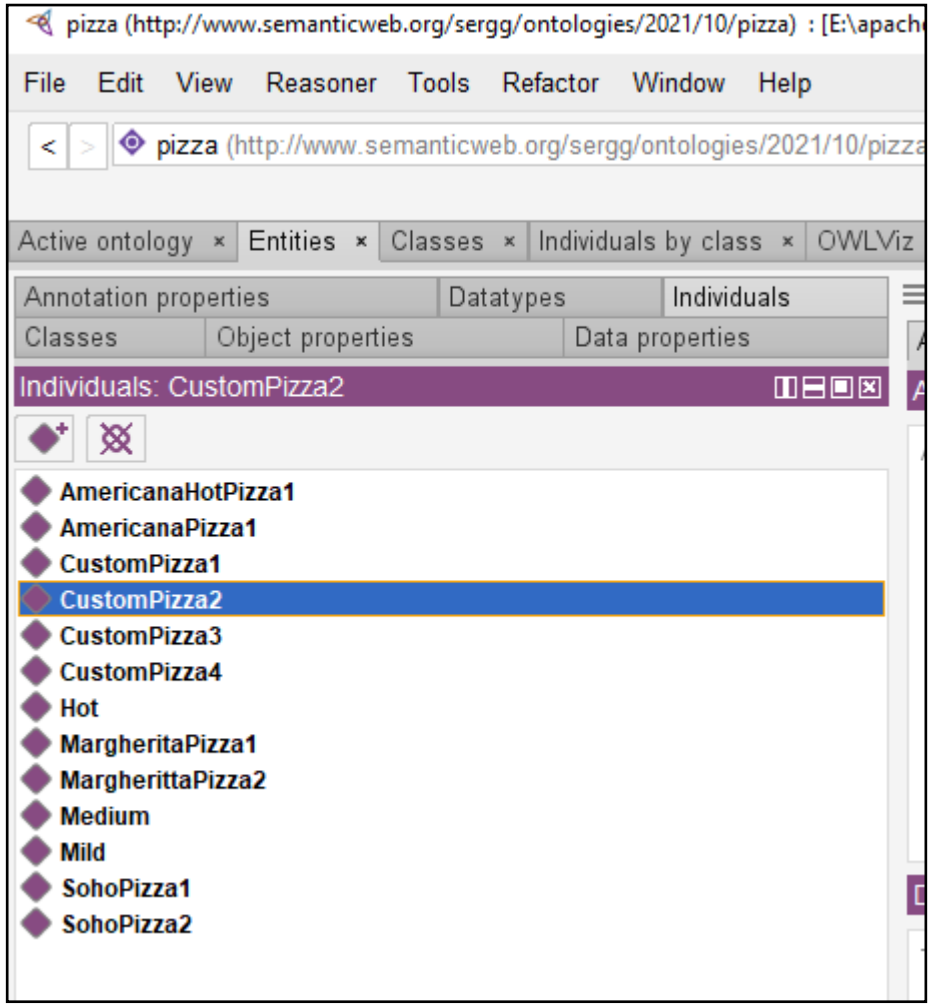
- Ir a Window > Tabs > Individuals by class
  - Añadir la vista Direct instances en Window > Views > Individual views > Direct instances
  - Hacer clic en el rombo morado de Direct instances para añadir individuos y, una vez seleccionados, poblar los datos que se deseen en la vista: Property assertions. En este caso vamos a poblar para cada uno “Data property assertions +”

- Al asignar un valor a la propiedad de datos hasCaloricContent, especificar el tipo de dato xsd:integer.


pizza (http://www.semanticweb.org/sergg/ontologies/2021/10/pizza) : [E:\apache-tomcat-10.0.11\webapps\pizza-lab-checkpoint-14.owl]
File Edit View Reasoner Tools Refactor Window Help
pizza (http://www.semanticweb.org/sergg/ontologies/2021/10/pizza)
Pizza NamedPizza MargheritaPizza
Active ontology Entities Classes Individuals by class OWLViz DL Query OntoGraf SWRLTab SQWRLTab
Class hierarchy: MargheritaPizza
Asserted
owl:Thing
SpicyPiz...
Pizza
InterestingPizza
NamedPizza
AmericanaHotPizza
AmericanaPizza
CheesyPizza
MargheritaPizza
SohoPizza
VegetarianPizza
PizzaBase
PizzaTopping
Spiciness
Annotations Usage Individuals by type (inferred)
Annotations: Mild
Annotations
Create a new Named individual
Name MargheritaPizza1
IRI http://www.semanticweb.org/sergg/ontologies/2021/10/pizza#MargheritaPizza1
New entity options...
Aceptar
Cancelar
Direct instances:
For: MargheritaPizza
Different Individuals
Negative data property assertions


pizza (http://www.semanticweb.org/sergg/ontologies/2021/10/pizza) : [E:\apache-tomcat-10.0.11\webapps\pizza-lab-checkpoint-14.owl]
File Edit View Reasoner Tools Refactor Window Help
pizza (http://www.semanticweb.org/sergg/ontologies/2021/10/pizza)
Pizza NamedPizza MargheritaPizza
Active ontology Entities Classes Individuals by class OWLViz DL Query OntoGraf SWRLTab SQWRLTab
Class hierarchy: MargheritaPizza
Asserted
owl:Thing
SpicyPiz...
Pizza
InterestingPizza
NamedPizza
AmericanaHotPizza
AmericanaPizza
CheesyPizza
MargheritaPizza
SohoPizza
VegetarianPizza
PizzaBase
PizzaTopping
Spiciness
Annotations Usage Individuals by type (inferred)
Annotations: MargheritaPizza1
Annotations
Description: MargheritaPizza1
Types
MargheritaPizza
Same Individual As
Property assertions: MargheritaPizza1
Object property assertions
Data property assertions
Negative object property assertions
Direct instances: MargheritaPizza1
For: MargheritaPizza
MargheritaPizza1
MargheritaPizza1
Data Property
Asserted
owl:topDataProperty
hasCaloricContent
345
Type xsd:integer
Lang
Aceptar
Cancelar
Reasoner state out of sync with active ontology
Show Inferences

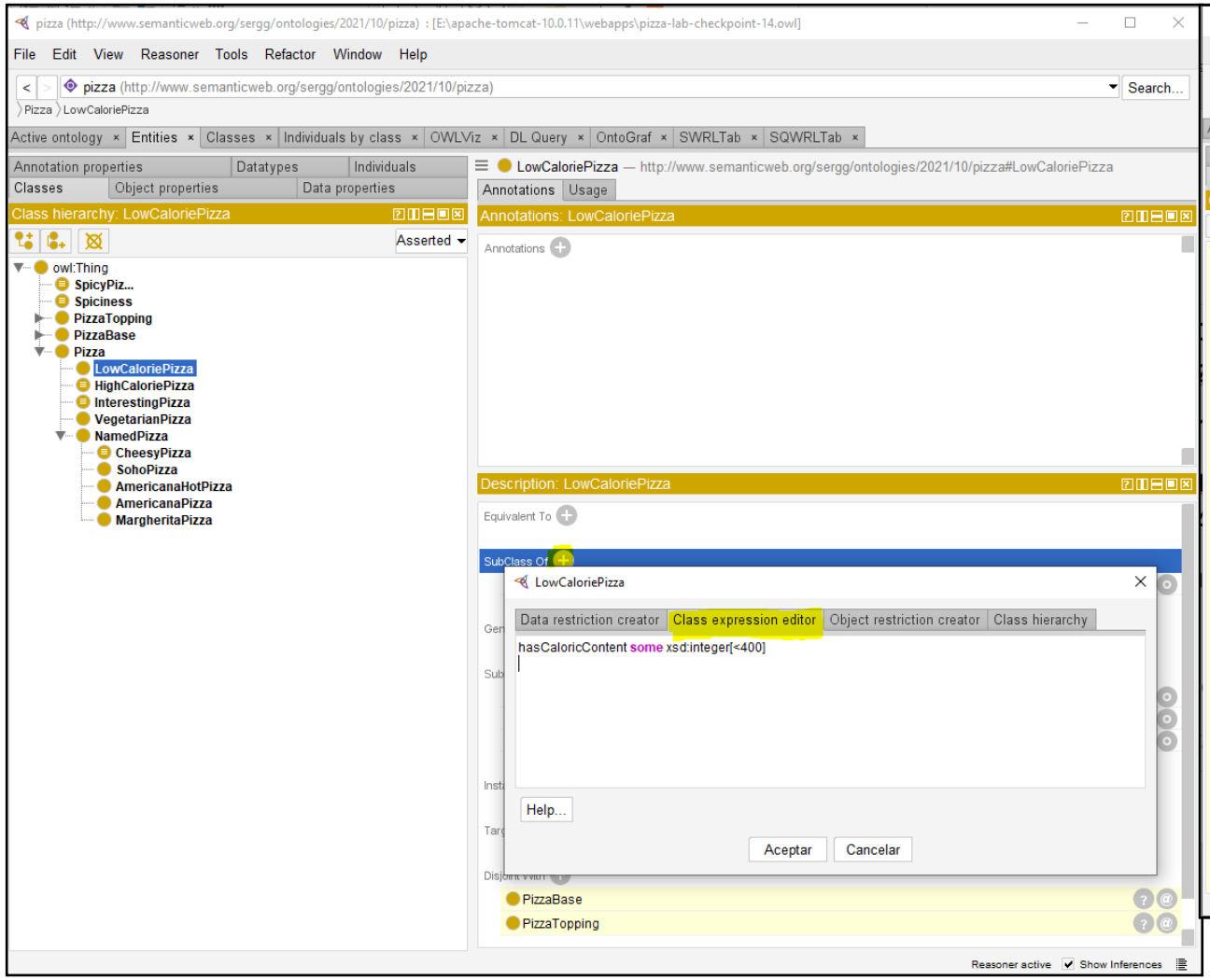

pizza (http://www.semanticweb.org/sergg/ontologies/2021/10/pizza) : [E:\apache-tomcat-10.0.11\webapps\pizza-lab-checkpoint-14.owl]
File Edit View Reasoner Tools Refactor Window Help
pizza (http://www.semanticweb.org/sergg/ontologies/2021/10/pizza)
Pizza
Active ontology × Entities × Classes × Individuals by class × OWLViz × DL Query × OntoGraf × SWRLTab × SQWRLTab ×
Class hierarchy: Pizza
Asserted
owl:Thing
SpicyPiz...
Pizza
InterestingPizza
NamedPizza
AmericanaHotPizza
AmericanaPizza
CheesyPizza
MargheritaPizza
SohoPizza
VegetarianPizza
PizzaBase
PizzaTopping
Spiciness
Direct instances: CustomPizza1
For: Pizza
CustomPizza1
Annotations Usage Individuals by type (inferred)
Annotations: CustomPizza1
Annotations
Description: CustomPizza1
Types
Pizza
Same Individual As
Different Individuals
Property assertions: CustomPizza1
Object property assertions
Data property assertions
hasCaloricContent 560
Negative object property assertions
Negative data property assertions

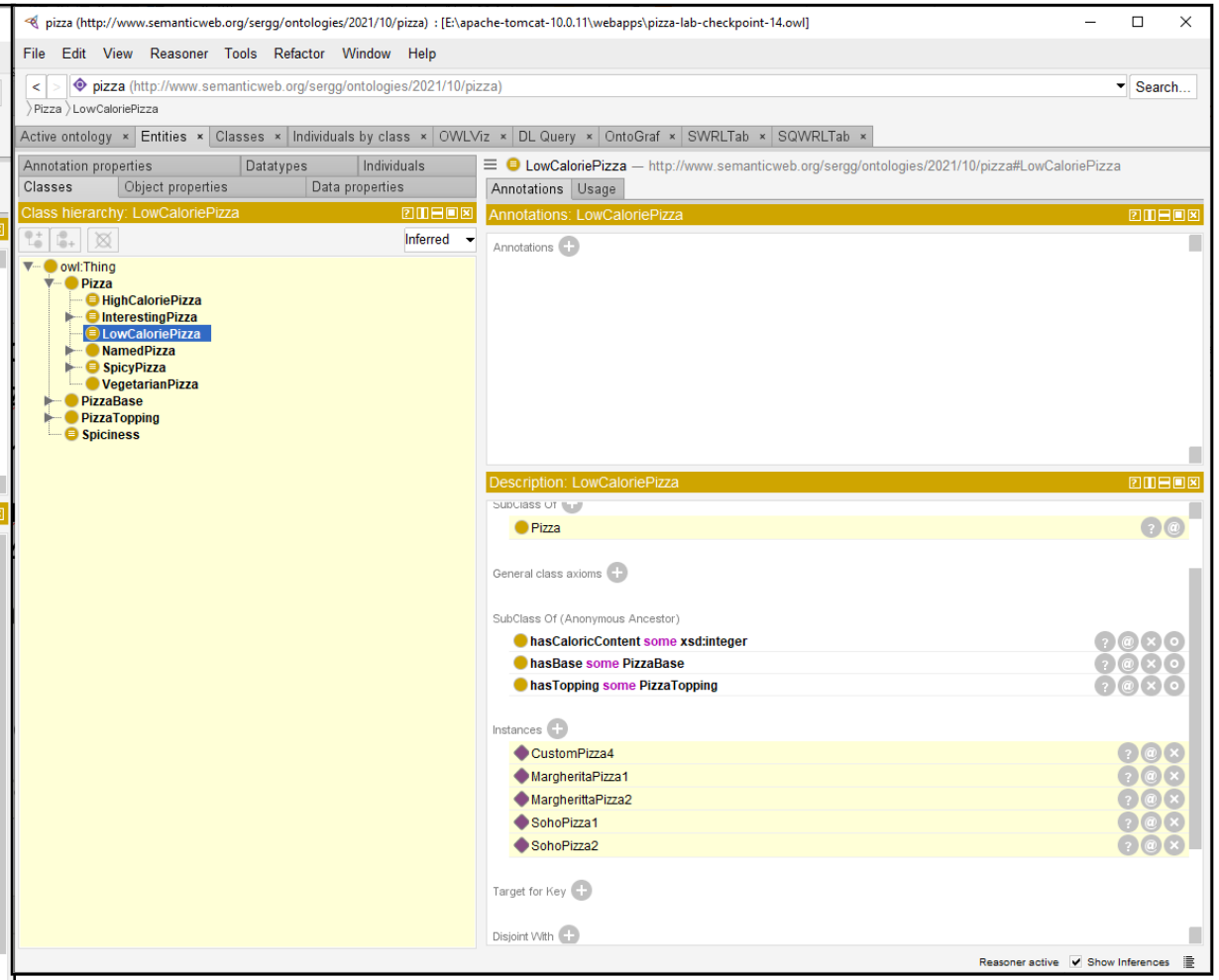

pizza (http://www.semanticweb.org/sergg/ontologies/2021/10/pizza) : [E:\apach
File Edit View Reasoner Tools Refactor Window Help
pizza (http://www.semanticweb.org/sergg/ontologies/2021/10/pizza
Active ontology × Entities × Classes × Individuals by class × OWLViz
Annotation properties Datatypes Individuals
Classes Object properties Data properties
Individuals: CustomPizza2
AmericanaHotPizza1
AmericanaPizza1
CustomPizza1
CustomPizza2
CustomPizza3
CustomPizza4
Hot
MargheritaPizza1
MargherittaPizza2
Medium
Mild
SohoPizza1
SohoPizza2

Definimos una clase que identifique a las Pizzas con alto nivel calórico

- Desde la vista de clases, crear una subclase de Pizza llamada HighCaloriePizza
- Seleccionar la nueva clase, y hacer click en SubClass Of +
- En el Class expression editor, introducir: hasCaloricContent some xsd:integer[>=400]
- Convertir la clase HighCaloriePizza en clase definida.

- Repetir los pasos para LowCaloriePizza, con valor menor de 400.
  - En este caso es conveniente dejar espacios entre los corchetes y la expresión “< 400”, si no, el editor puede dar un error.
- Sincronizar el razonador y observar los resultados inferidos





Checkpoint 14

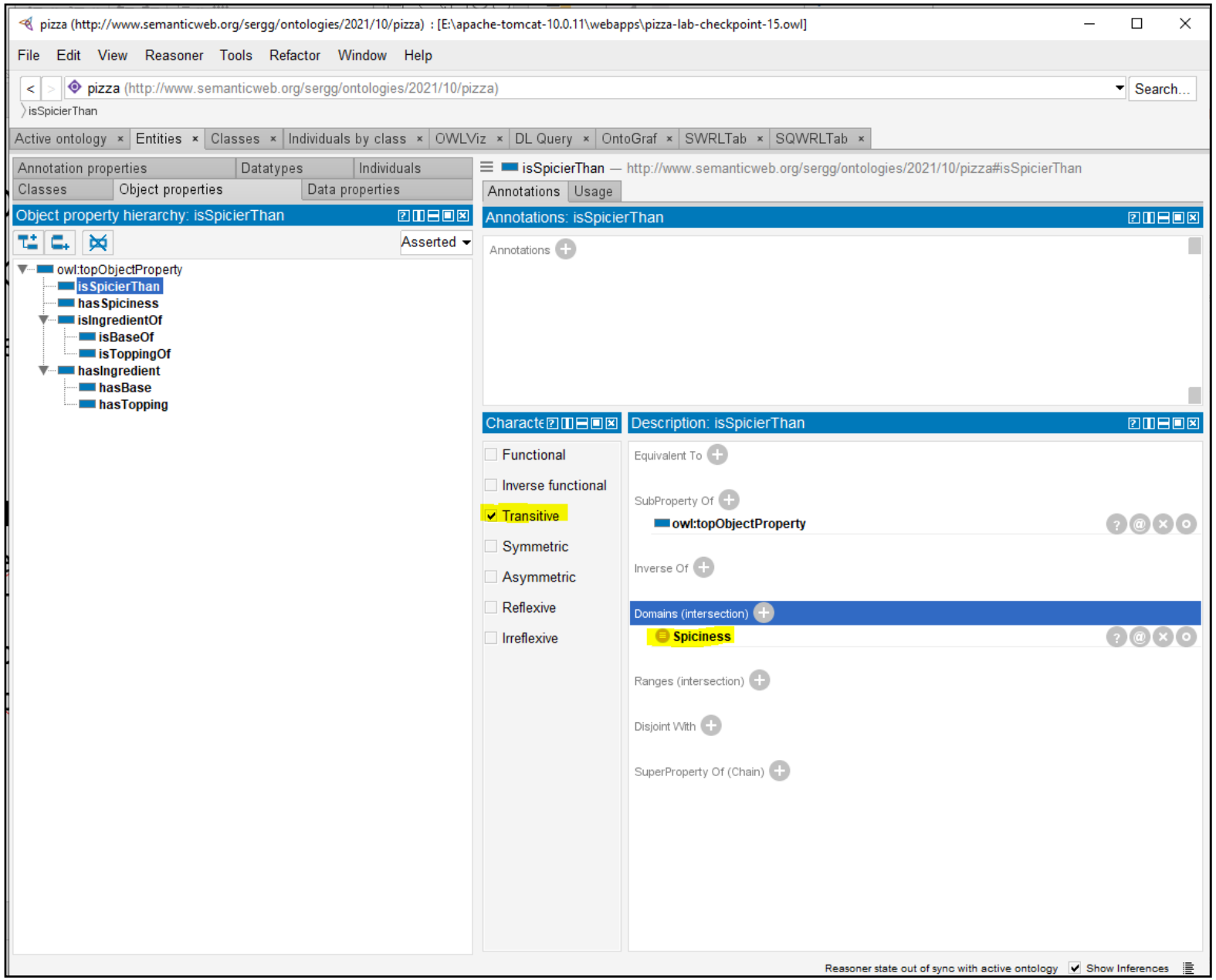
Graber y Sincronizar Razonador

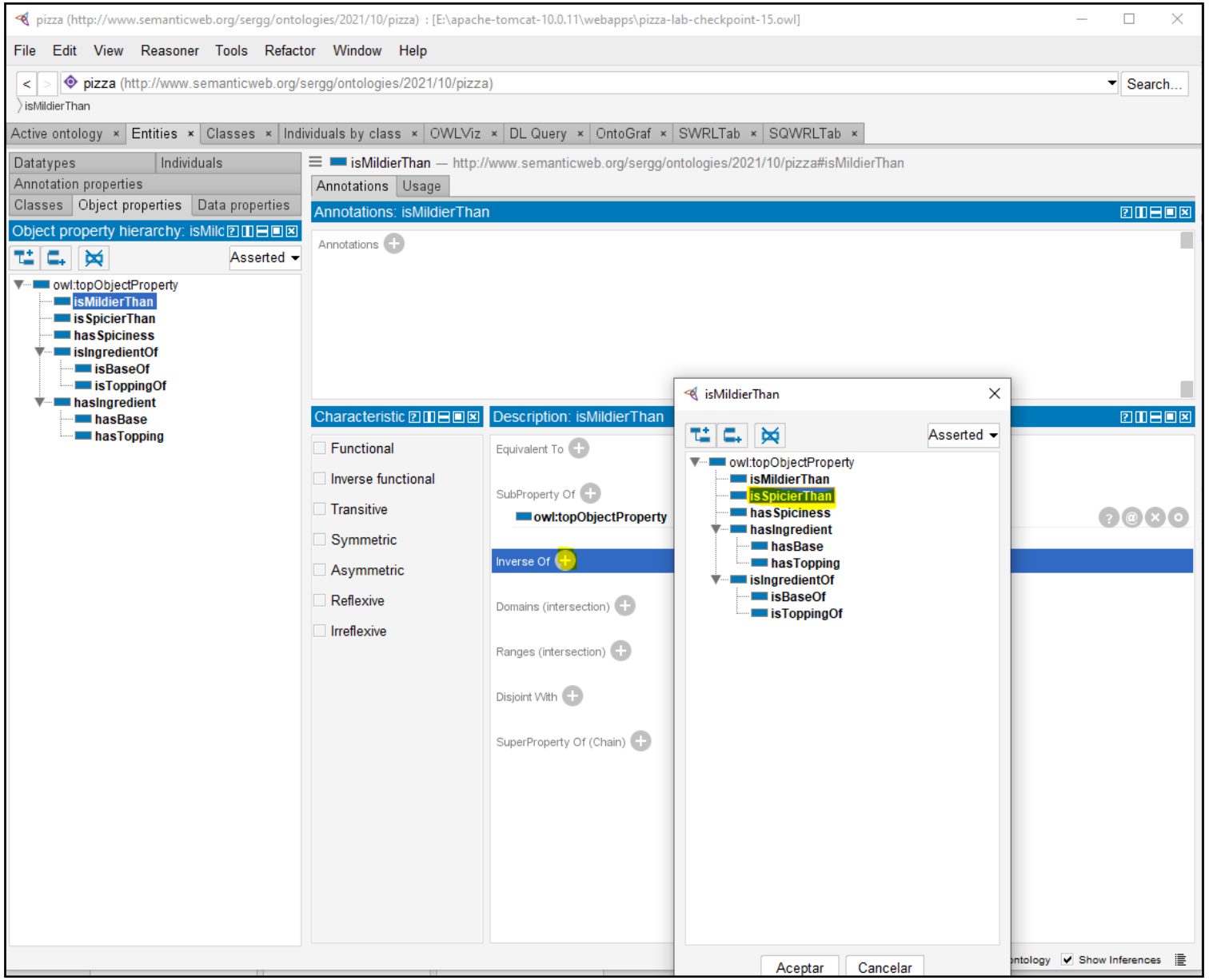
Definición de propiedades transitivas

Añadiendo Orden a una clase Enumerada mediante propiedades transitivas

- Ejercicio para demostrar una propiedad transitiva y transitiva inversa
- Definiremos un orden mediante propiedades de objetos para {Hot, Medium, Mild}

- Crear la propiedad: isSpicierThan bajo owl:topObjectProperty
- Domain: Spiciness
- Características: Transitiva
- Creamos la propiedad inversa: isMilderThan
- Inverse Of + isSpicerThan
- Sincronizar el razonador y comprobar las inferencias que se han generado gracias a las características definidas.

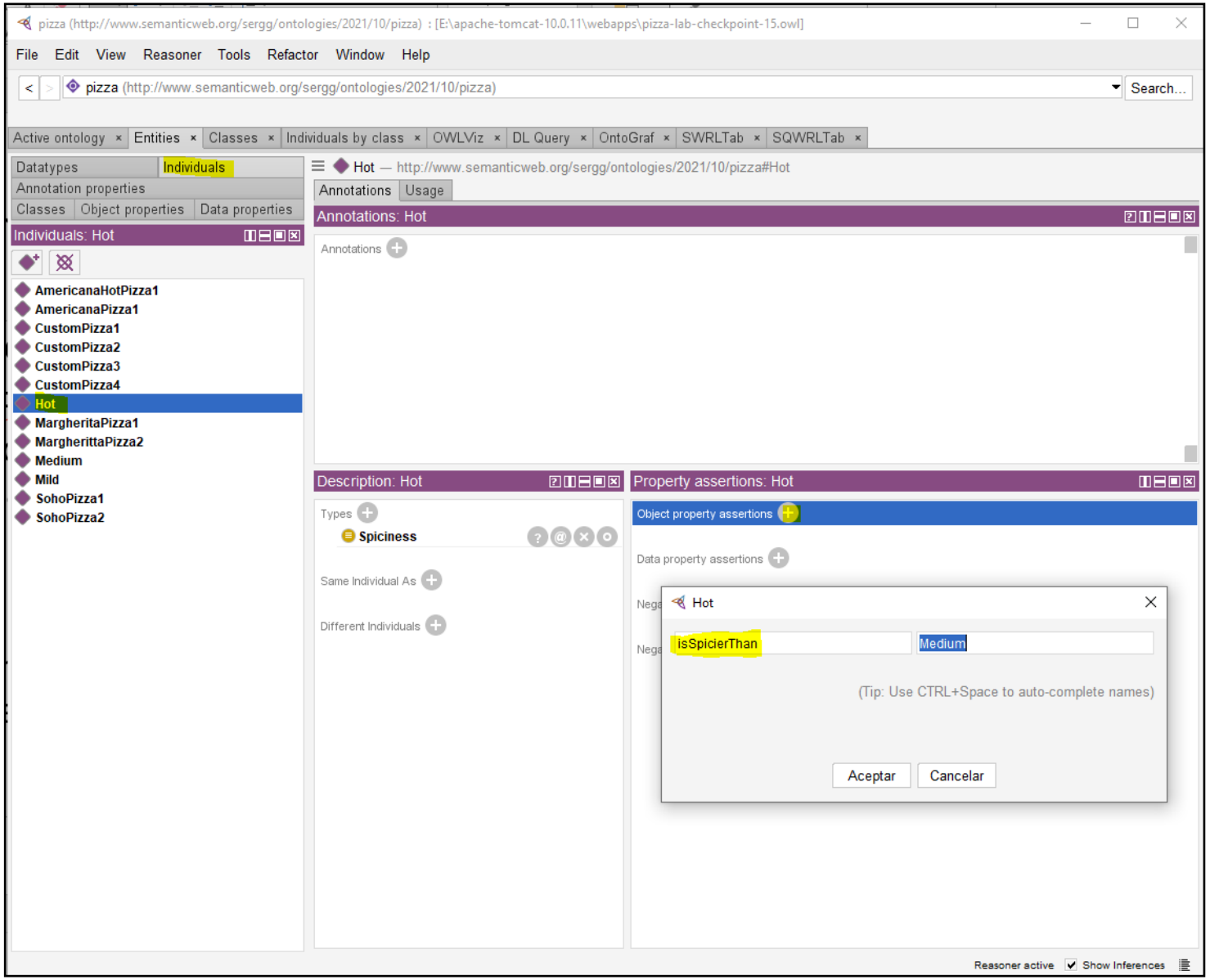

pizza (http://www.semanticweb.org/sergg/ontologies/2021/10/pizza) : [E:\apache-tomcat-10.0.11\webapps\pizza-lab-checkpoint-15.owl]
File Edit View Reasoner Tools Refactor Window Help
pizza (http://www.semanticweb.org/sergg/ontologies/2021/10/pizza)
Search...
isSpicierThan
Active ontology Entities Classes Individuals by class OWLViz DL Query OntoGraf SWRLTab SQWRLTab
Annotation properties Datatypes Individuals
Classes Object properties Data properties
Object property hierarchy: isSpicierThan
Asserted
owl:topObjectProperty
isSpicierThan
hasSpiciness
isIngredientOf
isBaseOf
isToppingOf
hasIngredient
hasBase
hasTopping
isSpicierThan — http://www.semanticweb.org/sergg/ontologies/2021/10/pizza#isSpicierThan
Annotations Usage
Annotations: isSpicierThan
Annotations
Characte
Description: isSpicierThan
Functional
Inverse functional
Transitive
Symmetric
Asymmetric
Reflexive
Irreflexive
Equivalent To
SubProperty Of
owl:topObjectProperty
Inverse Of
Domains (intersection)
Spiciness
Ranges (intersection)
Disjoint With
SuperProperty Of (Chain)
Reasoner state out of sync with active ontology
Show Inferences

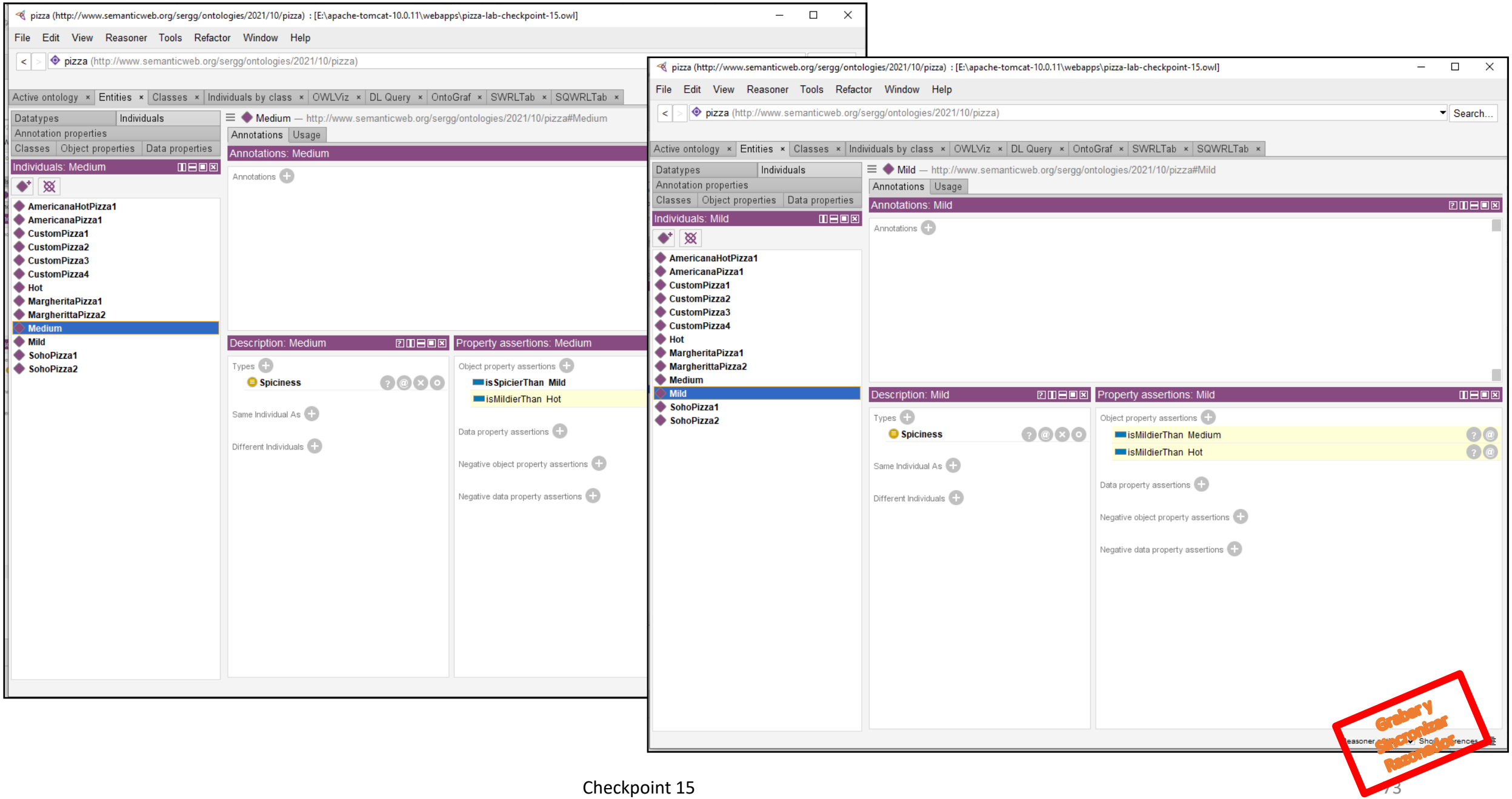

pizza (http://www.semanticweb.org/sergg/ontologies/2021/10/pizza) : [E:\apache-tomcat-10.0.11\webapps\pizza-lab-checkpoint-15.owl]
File Edit View Reasoner Tools Refactor Window Help
pizza (http://www.semanticweb.org/sergg/ontologies/2021/10/pizza)
Search...
isMildierThan
Active ontology Entities Classes Individuals by class OWLViz DL Query OntoGraf SWRLTab SQWRLTab
Datatypes Individuals
Annotation properties
Classes Object properties Data properties
Object property hierarchy: isMilc
Asserted
owl:topObjectProperty
isMildierThan
isSpicierThan
hasSpiciness
isIngredientOf
isBaseOf
isToppingOf
hasIngredient
hasBase
hasTopping
isMildierThan — http://www.semanticweb.org/sergg/ontologies/2021/10/pizza#isMildierThan
Annotations Usage
Annotations: isMildierThan
Annotations
Characteristic
Functional
Inverse functional
Transitive
Symmetric
Asymmetric
Reflexive
Irreflexive
Description: isMildierThan
Equivalent To
SubProperty Of
owl:topObjectProperty
Inverse Of
Domains (intersection)
Ranges (intersection)
Disjoint With
SuperProperty Of (Chain)
isMildierThan
Asserted
owl:topObjectProperty
isMildierThan
isSpicierThan
hasSpiciness
hasIngredient
hasBase
hasTopping
isIngredientOf
isBaseOf
isToppingOf
Aceptar
Cancelar
Show Inferences

Definiendo el orden de la propiedad

- En la vista de Individuals, seleccionar Hot.
- En Object Property assertions, definimos “isSpicerThan”, y a continuación introducimos el valor Medium, que es el siguiente inmediato a Hot.
- Repetimos para definir que Medium es más picante que Mild.

- Sincronizamos el razonador y observamos como se han añadido las nuevas inferencias (ver dos transparencias más adelante)

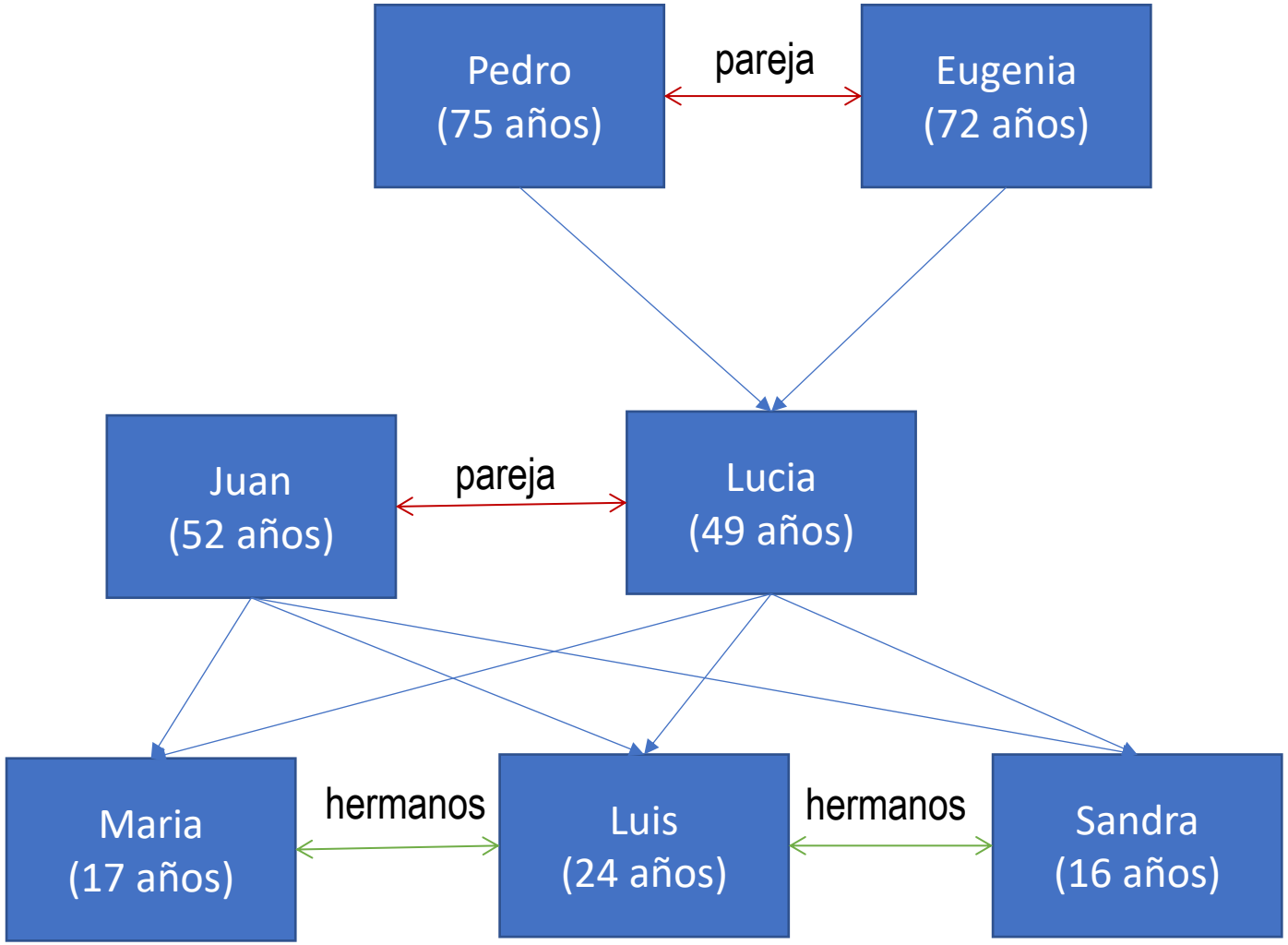

pizza (http://www.semanticweb.org/sergg/ontologies/2021/10/pizza) : [E:\apache-tomcat-10.0.11\webapps\pizza-lab-checkpoint-15.owl]
File Edit View Reasoner Tools Refactor Window Help
pizza (http://www.semanticweb.org/sergg/ontologies/2021/10/pizza)
Search...
Active ontology Entities Classes Individuals by class OWLViz DL Query OntoGraf SWRLTab SQWRLTab
Datatypes Individuals
Annotation properties
Classes Object properties Data properties
Individuals: Hot
AmericanaHotPizza1
AmericanaPizza1
CustomPizza1
CustomPizza2
CustomPizza3
CustomPizza4
Hot
MargheritaPizza1
MargherittaPizza2
Medium
Mild
SohoPizza1
SohoPizza2
Hot — http://www.semanticweb.org/sergg/ontologies/2021/10/pizza#Hot
Annotations Usage
Annotations: Hot
Annotations
Description: Hot
Types
Spiciness
Same Individual As
Different Individuals
Property assertions: Hot
Object property assertions
Data property assertions
Hot
isSpicierThan
Medium
(Tip: Use CTRL+Space to auto-complete names)
Aceptar
Cancelar
Reasoner active
Show Inferences



Checkpoint 15

PRÁCTICA: PARTE 2

- Completar la ontología sobre las familias, considerando:
  - Definir la edad de una persona
  - Definir el sexo de la persona como una clase enumerada
  - Especificar subpropiedades de esDescendienteDe y esAscendienteDe en función del sexo (esPadreDe, esMadreDe, esHijoDe, esHijaDe)
  - Crear una clase para identificar a las personas menores de edad y otra para mayores de 65 años
  - Una clase que identifique a las personas que estén emparejadas y que tengan descendencia
  - Una clase que identifique a los padres y otra a las madres
  - Lo que consideres que enriquezca la ontología


Pedro
(75 años)
pareja
Eugenia
(72 años)
Juan
(52 años)
pareja
Lucia
(49 años)
Maria
(17 años)
hermanos
Luis
(24 años)
hermanos
Sandra
(16 años)

FIN DE LA SEGUNDA SESION